

To be covered over two weeks

Lecture 3: Authenticity (Data Origin: mac & signature)

3.1. Crypto Primitive: Public Key Cryptography

3.1.1 RSA

3.1.2 Security of RSA

3.1.3 Remarks

3.2 Data Authenticity (Hash): Unkeyed hash

3.3 Data Authenticity (Mac) : Keyed Hash

3.4 Data Authenticity (Signature): Asymmetric key.

3.5 Some attacks & pitfalls

3.5.1 Birthday attack on hash

3.5.2 Design flaw: using encryption for authenticity

3.5.3 Time-Space tradeoff

3.6 Application of Hash: password file protection (in Lecture 2)

Summary & takeaways

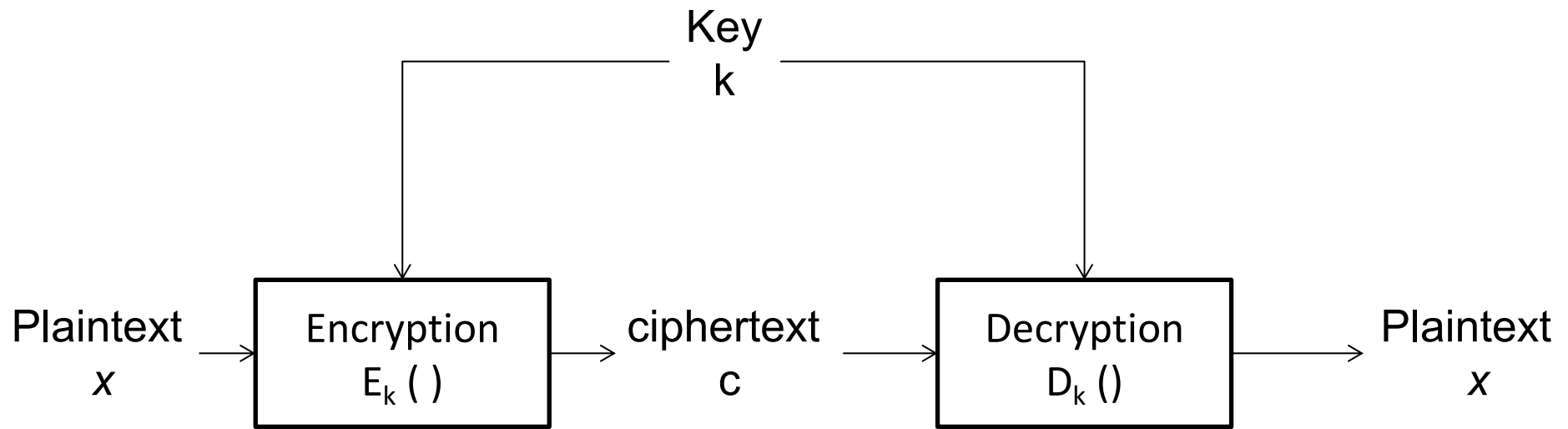
- Public Key Encryption*.
 - RSA (based on integer factorization. Only integer). ElGamal (based on discrete log of “Algebraic group”. Many choices: ECC).
 - Differences with symmetric key. Implications:
 - Symmetric key requires a **secure channel** to distribute key.
 - Public key requires a **secure broadcast channel** to distribute key.
 - Pitfall: using RSA in symmetric key setting.
- Authentication crypto primitives: digest, mac, signature.
 - Security models and application scenarios (Slide 71,72,73)
 - No key or other secret information in hash/digest.
 - Advantages of signature: public key setting (broadcast channel); non-repudiation.
 - Disadvantages of signature: efficiency.
- Achieve Confidentiality $\nRightarrow$ Achieve Authenticity
- Various construction: SHA, HMAC, CBC-mac, DSA, hash-and-sign (internal mechanism not required).
- Birthday attacks for collision. (analogous to exhaustive search in encryption)
- Time-memory-tradeoff in inverting hash

3.1. Crypto Primitive: PKC

We have seen a crypto primitive: symmetric key encryption. Now, we are going to introduce a few more. *PKC, Hash, Key-ed Hash (mac), Signature.*

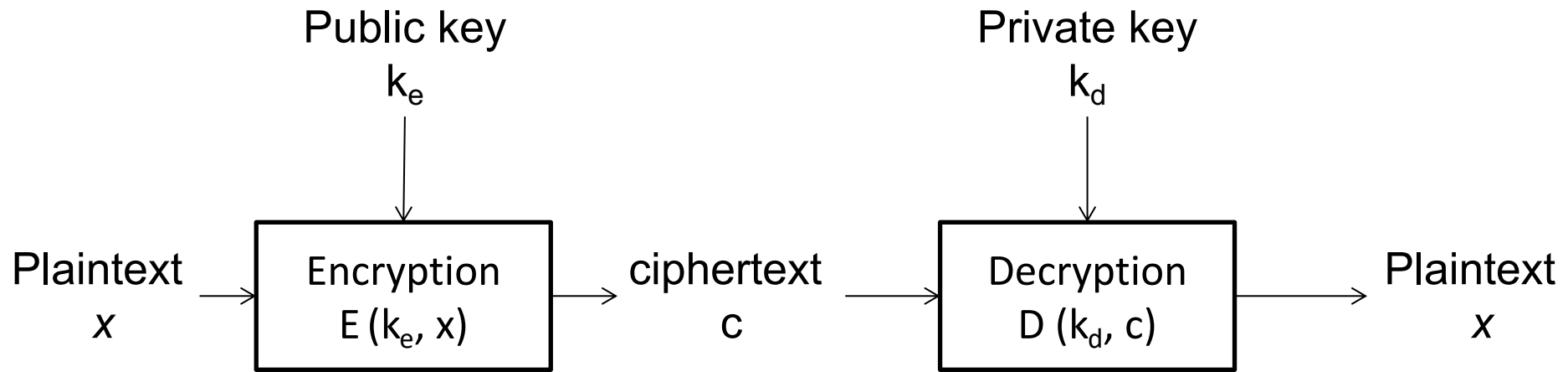
Overview

A symmetric-key encryption scheme uses the same key for encryption and decryption.



Overview

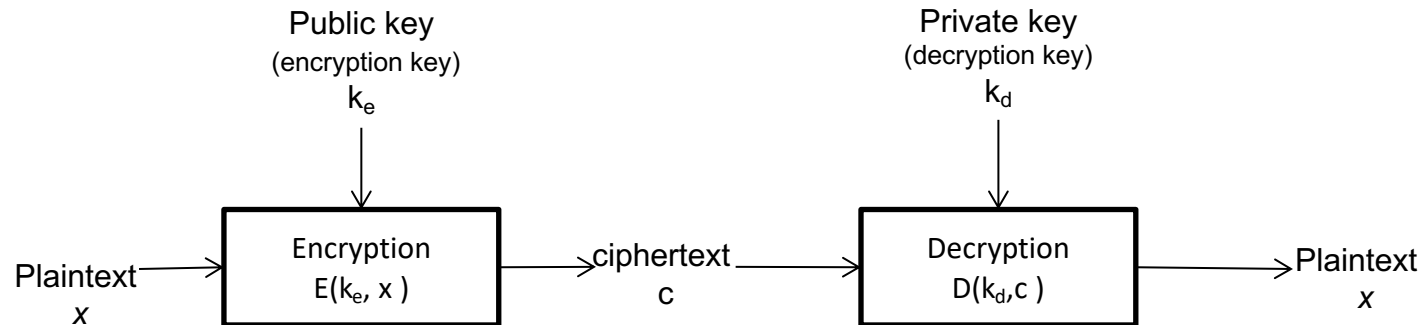
A public key (aka asymmetric-key) scheme uses different keys for encryption and decryption.



Note:

- The private key is typically of the form: $k_d = \langle k_1, k_2 \rangle$ where k_1 is the public key. That is, part of the private key is the public key.
- Very often, we also use this definition: we call k_2 the private key, k_1 the public key and define the decryption key as both public & private key. Just a different way to define the same the concept.
- In this module, we interchange the definition for convenience and should be clear in the context.

Why called “public key”?



In a typical usage, the owner Alice keeps the private key as a secret but tell everyone the public key (for eg. posts in her Facebook).

An entity, Bob, has a ***plaintext*** x for Alice. Bob can encrypt it (using the ***public key***) and posts the ***ciphertext*** c on Alice's Facebook.

Another entity, Eve, obtains the public key, and ciphertext c from the Alice's Facebook. Without the private key, Eve is unable to derive x .

Alice, with the ***private key***, can decrypt and obtain the plaintext x .

Security Requirements of a Public Key Scheme

- *security requirement*:
 - Given the public key and the ciphertext (but not the private key), it is difficult to determine the plaintext;
- The requirement implies that it must be difficult to get the private key from the public key.

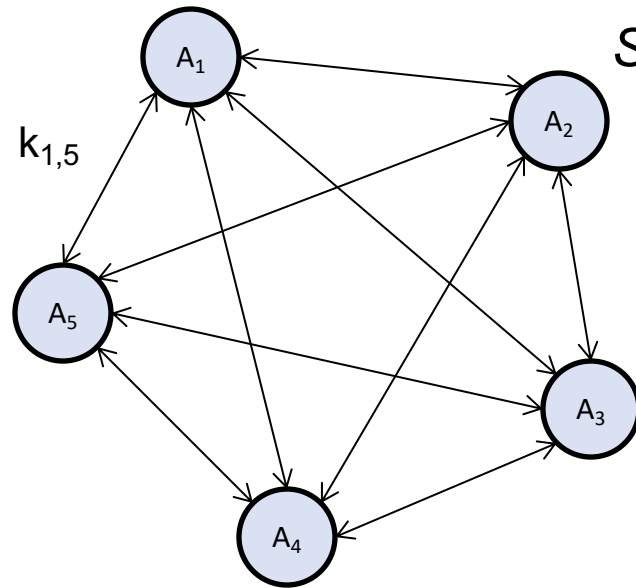
note:

- A refined notion of “*difficult to determine the plaintext*” is the indistinguishability of the ciphertext from a truly random source, i.e. to an entity who doesn’t know the private key, the ciphertext resembles a sequence of random values. Notion of indistinguishability would be covered in CS4236 (crypto).
- **Encryption Oracle**: Anyone (including attacker) can encrypt. So, the “encryption oracle” is always available to anyone. So, chosen plaintext attack is a must to be considered.

Advantages in Key Management: only requires Broadcast Channel

- Suppose we have multiple entities, $A_1, A_2, \dots, A_n$:
 - Each of them can compute a pair of <private key, public key>
 - Each broadcast his/her public key, but keeps the private key secret
- Now, suppose A_i wants to encrypt a message m to be read only by A_j :
 - A_i can use A_j 's public key to encrypt m
 - By the property of PKC, only A_j can decrypt it
- If we don't use the PKC, then, any two entities must share a symmetric key via a secure channel. Implications:
 - Many keys are required in symmetric keys;
 - Symmetric key requires both entities to know each other before the actual communication session. In contrast, PKC can handle such scenario. (e.g. You want to send a message to www.bbc.com. Does www.bbc.com know you? Likely no. If www.bbc.com doesn't know you, it is impossible to set up a secure channel for you and www.bbc.com to establish a shared symmetric key. Now, with PKC, www.bbc.com can first broadcasts its public key. Even an entity whom www.bbc.com doesn't know can get the public key.)

Keys Distribution



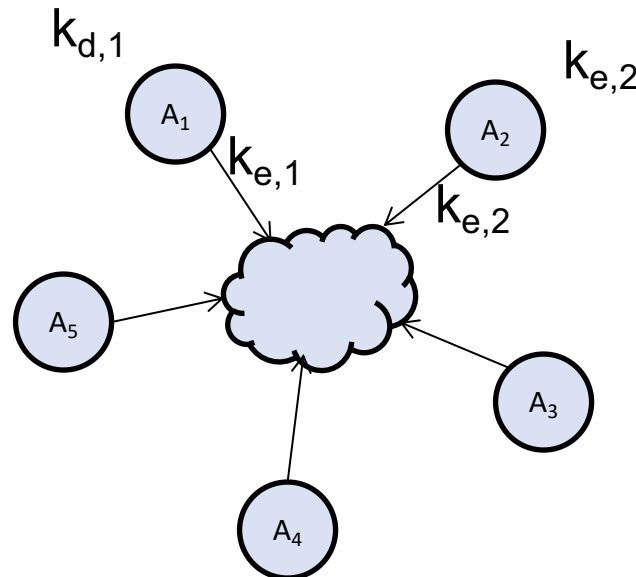
Symmetric key setting: **Individual secure channel**

Every pair of entities requires one key.

Let $k_{i,j}$ to be the key to be shared by A_i and A_j

$k_{1,2}, k_{1,3}, \dots$

Total number of keys: $n(n-1)/2$



Public key setting: **Secure Broadcast Channel**

Each entity publishes its public key

Entity A_i publish $k_{d,i}$ and keep $k_{e,i}$

Total number of public keys: n

Total number of private keys: n

Many books stress that PKC is “better” because it uses lesser keys. In my opinion, the main advantage is that:

the entities do not need to know each other before broadcasting the public keys.

Important note

- It is wrong to say that in PKC, we no longer need to distribute key. We still need a secure broadcast channel to distribute the public key.

(whole lecture on PKI)

- While we are using encryption to describe PKC, an important application of PKC is in authentication (e.g. HTTPS).

Popular PKC schemes

- RSA Key size ≈ 2048 bits
- ElGamal: ElGamal can exploit techniques in Elliptic Curve Cryptography (ECC), reducing the key size to ≈ 300 bits.
- Paillier: Partial homomorphic with respect to addition.



The idea of an asymmetric public-private key cryptosystem is attributed to Whitfield Diffie and Martin Hellman, who published the concept in 1976.

Diffie and Hellman
2015 Turing Award



Ron Rivest, Adi Shamir, and Leonard Adleman proposed RSA algorithm in 1977.

Rivest, Shamir and Adleman,
2002 Turing Award.

Integer representations

- We will introduce the “classroom” RSA, i.e. the basic form of RSA. We call it “classroom” because the variant used in practice is different, with padding and special considerations in choosing the primes, etc. Also called “textbook” RSA.
- Many PKC represent the data (plaintext, ciphertext, key) as integers, and the algorithm are some arithmetic operations on the integers. The integers can be represented using binary representations. When we say that a key is of 1024-bit, we mean that the key can be represented using 1024 bits under binary representation. (e.g. a 3-bit integer is a value from 0 to 7)
- Note that the total number of 1024-bit integers are 2^{1024} . So, an algorithm that exhaustively searches 1024-bit numbers is infeasible.

3.1.1 RSA

Classroom/Textbook RSA: algo that taught in many “non-security” modules.

In practice: Padded RSA with strong primes. In some cases, secure implementation to guard against side-channel attack.

“Classroom RSA” - setup

1. Owner randomly chooses 2 large primes p, q and computes $n = pq$.
2. Owner randomly chooses an encryption exponent e s.t. $\gcd(e, (p-1)(q-1)) = 1$. (i.e. e does not have common factor with $(p-1)$ or $(q-1)$.)
3. Owner finds the decryption exponent d

$$\text{where } de \bmod (p-1)(q-1) = 1$$

There is an algorithm that finds d , when given e, p and q . We won't get into the details,

The term $(p-1)(q-1) = \Phi(n)$ is aka the Euler's totient function, which is the number of co-primes $< n$.

4. Owner publishes (n, e) as public key, and safe-keep (n, d) as the private key. (note that owner doesn't need to keep p, q)

Encryption, Decryption

public key (n, e)

- **Encryption,**
Given m , the ciphertext c is

$$c = m^e \bmod n$$

private key (n, d)

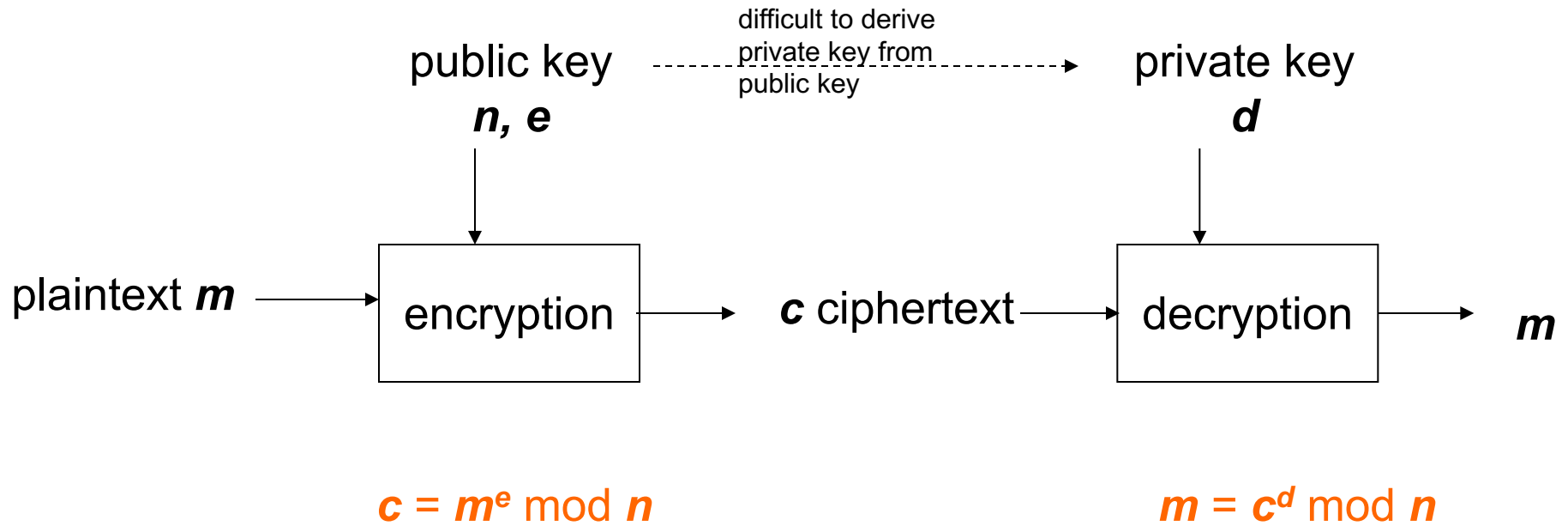
- **Decryption**
Given c , the plaintext m is

$$m = c^d \bmod n$$

RSA

$$n = pq$$
$$\Phi(n) = (p-1)(q-1)$$

$$d = e^{-1} \bmod \Phi(n)$$



Optional remark: Consider the relationship: $c = m^e \bmod n$

- (RSA problem) Given c, n, e , find m , (called the e -th root of c)
- (discrete log problem): Given c, n, m , find e , (called the discrete log of c)

Correctness of RSA

- Note that for any positive $m < n$, and any pair of public/private keys,

$$\text{Decrypt}(\text{Encrypt}(m)) = m$$

That is,

$$(m^e)^d \bmod n = m$$

Optional

Proof (sketch): The correctness depends on this property of modulo:

For any m, r, n , we have

$$m^r \bmod n = m^{r \bmod \Phi(n)} \bmod n$$

(when n is product of two primes, p, q , then $\Phi(n) = (p-1)(q-1)$.)

Now, combining the fact that

$$de \bmod (p-1)(q-1) = 1,$$

we have $m^{de} \bmod n = m^1 \bmod n = m$

Example

- $p=5, q=11, n=55$

- $(p-1)(q-1) = 40$

- Suppose $e = 3$, then $d = 27$

There is an efficient algorithm
that computes d from p, q, e . Details omitted.

note that indeed $3 \times 27 \bmod 40 = 81 \bmod 40 = 1$

- Suppose $m = 9$

Encrypt:

$$c = m^e \bmod n = 9^3 \bmod 55 = 14$$

Decrypt

$$c^d \bmod n = 14^{27} \bmod 55$$

$$= 14^{8 \times 2 + 8 + 3} \bmod 55$$

$$= (36 \times 16 \times 49) \bmod 55$$

$$= 9$$

There is an efficient algorithm
that computes modulo exponentiation.
Details omitted.

Interchangeable role of encryption and decryption key

- Classroom RSA has this property: we can also use the decryption key d to encrypt, and then the encryption key e to decrypt. In other words, we can swap the d and e .
- The above property is special in RSA. It usually does not hold in other public key schemes. For e.g. the ElGamal PKC (not covered in this module) doesn't has this property. This property is useful in designing signature scheme.

Warning: Some documents flip encryption/decryption when describing RSA, i.e. they use public key to decrypt, and private key to encrypt. This could be very confusing. The “source” of this inconsistencies is due to how we use RSA in a signature scheme. In RSA-based signature, the decryption key is broadcast and everyone know the decryption key. So, the decryption key is the “public” key.

Algorithmic issues

- (Encryption/decryption) Given n, m, e , there is an efficient algo to compute exponentiation: $m^e \bmod n$. Likewise, there is an efficient algo to compute $c^d \bmod n$

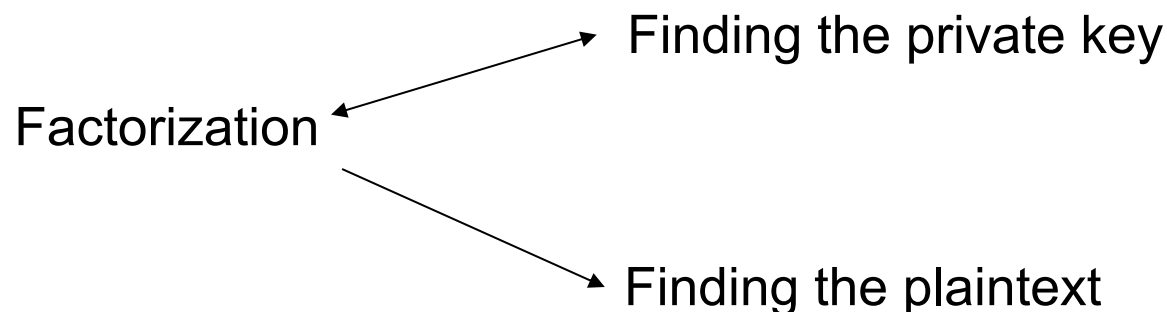
(To get additional speedup, choose a small e so that encryption can be done very fast. It turns out that the value of e won't affect the security. But it can't be too small due to some attack. Very often, we choose e to be 65537.)

- (step 1 in setup: Primality test) How to find a random prime? Randomly pick a number, and test whether it is a prime. Since there are many primes, there is a high chance that a randomly chosen number is prime. (as we shall see later, some primes are weak and lead to attack. Using "strong prime" to avoid those attacks)
- (step 3 in setup) The value of d can be efficiently computed from e and n using the *extended Euclidean algorithm*.

3.1.2 Security of RSA

Security of RSA

- It can be shown that, the problem of getting the RSA private key from public key is as difficult as the problem of factorizing n .
- However, it is not known whether the problem of getting the plaintext from the ciphertext is as difficult as factorization, that is, it could be easier.



State-of-the-art on factorization

- more info: http://en.wikipedia.org/wiki/Integer_factorization
- A 640 bits number was successfully factored in Nov 2, 2005, using approximately 30 2.2GHz-Opteron-CPU years. It is computed over five months using multiple machines.
- A 768 bits number (RSA-768) was factored in Dec 2009, using hundreds of machines over 2 years.
- See NIST recommendation.

- Here is the 640-bits number

31074182404900437213507500358885679300373460228427
27545720161948823206440518081504556346829671723286
78243791627283803341547107310850191954852900733772
4822783525742386454014691736602477652346609

=

16347336458092538484431338838650908598417836700330
92312181110852389333100104508151212118167511579

*

1900871281664822113126851573935413975471896789968
515493666638539088027103802104498957191261465571

- Here is the 768-bits number

123018668453011775513049495838496272077285356959533479219732245
215172640050726365751874520219978646938995647494277406384592519
255732630345373154826850791702612214291346167042921431160222124
0479274737794080665351419597459856902143413

=

347807169895689878604416984821269081770479498371376856891243138
8982883793878002287614711652531743087737814467999489

*

367460436667995904282446337996279526322791581643430876426760322
83815739666511279233373417143396810270092798736308917

from

T Kleinjung et al., *Factorization of a 768-bit RSA modulus*, eprint 2010.

<https://eprint.iacr.org/2010/006.pdf>

post-Quantum cryptography

- A Quantum computer can factorize and perform “*discrete log*” in polynomial time. In 2001, a 7-*qubits* quantum computer was built to factor 15, carried out by IBM using NMR.

http://domino.watson.ibm.com/comm/pr.nsf/pages/news.20011219_quantum.html

Hence, both RSA and “discrete log based” PKC will be broken with Quantum computer.

Post-Quantum Cryptography: This generally refers to PKC that are secure against quantum computer.

- Lattice-based cryptography: Based on this hard problem-- Given the “basis” of lattice, it is computationally hard to find the shortest (or approx) lattice point. Many proposals, no clear winner yet.
- Multivariate polynomial: Given a multivariate polynomial, it is difficult to find the solution (under modulo p). No secure construction yet.

Padding of RSA

- Same as symmetric-key encryption, some forms of IV is required so that encryption of the **same** plaintext at different time would give different ciphertext. Hence additional mechanisms are added to the “classroom” RSA.
- Classroom RSA has some interesting properties, e.g. “homomorphic property”. These properties are useful in applications (e.g. blind signature, encrypted domain processing), but they lead to attacks and information leakages. Such properties can be destroyed using padding.
- The standard (Public-Key Cryptography Standards) PKCS#1, add “optimal padding” to achieve the above http://en.wikipedia.org/wiki/PKCS_1

Pitfall: (CWE-780) Using classroom RSA

<https://cwe.mitre.org/data/definitions/780.html>

▼ Description

The software uses the RSA algorithm but does not incorporate Optimal Asymmetric Encryption Padding (OAEP), which might weaken the encryption.

▼ Extended Description

Padding schemes are often used with cryptographic algorithms to make the plaintext less predictable and complicate attack efforts. The OAEP scheme is often used with RSA to nullify the impact of predictable common text.

▼ Relationships

The table(s) below shows the weaknesses and high level categories that are related to this weakness. These relationships are defined as ChildOf, ParentOf, MemberOf and give insight to similar items that may exist at higher and lower levels of abstraction. In addition, relationships such as PeerOf and CanAlsoBe are defined to show similar weaknesses that the user may want to explore.

▼ Likelihood Of Exploit

Medium

▼ Demonstrative Examples

Example 1

The example below attempts to build an RSA cipher.

Example Language: **Java**

(bad code)

```
public Cipher getRSACipher() {
    Cipher rsa = null;
    try {
        rsa = javax.crypto.Cipher.getInstance("RSA/NONE/NoPadding");
    }
    catch (java.security.NoSuchAlgorithmException e) {
        log("this should never happen", e);
    }
    catch (javax.crypto.NoSuchPaddingException e) {
        log("this should never happen", e);
    }
    return rsa;
}
```

While the previous code successfully creates an RSA cipher, the cipher does not use padding. The following code creates an RSA cipher using OAEP.

Example Language: **Java**

(good code)

```
public Cipher getRSACipher() {
    Cipher rsa = null;
    try {
        rsa = javax.crypto.Cipher.getInstance("RSA/ECB/OAEPWithMD5AndMGF1Padding");
    }
    catch (java.security.NoSuchAlgorithmException e) {
        log("this should never happen", e);
    }
    catch (javax.crypto.NoSuchPaddingException e) {
        log("this should never happen", e);
    }
    return rsa;
}
```

Issue 2 (Security of RSA)

RSA is not necessary “more secure” than AES.

- One common argument that RSA is better than AES: *RSA is “provably” secure*. While it can be proven that, getting ***the private key from the public key*** is as difficult as factorization, but it is still not known whether the problem of getting ***the plaintext from the ciphertext and public key*** is as difficult as factorization (this is known as the RSA problem).
- The “classroom” RSA needs to be modified to prevent some attacks. Many attacks exploit one nice property of RSA: ***homomorphic property***. This property is a double-edged sword: it is useful (e.g. blind signature), but is also a vulnerability.
(optional) It turns out that the classroom RSA leaks one-bit of information about the plaintext! If adversary knows the module n , and knows the ciphertext c , the adversary can derive one “bit” of information regarding the plaintext. This is because RSA encryption preserves the “Jacobi symbol”, i.e. the Jacobi symbol of the plaintext and ciphertext is the same.
- Factorization can be efficiently done by Quantum Computer. So, RSA is broken under Quantum Computer. It is not clear whether Quantum Computer can break AES.

(Optional) Quantum computer can speedup exhaustive of AES key by a square root. That is, to exhaustively search 128-bit AES key, quantum computer take at most 2^{64} steps. This is call Grover’s search. While seems a lot of reduction, by doubling the key length to 256-bits, quantum computer would take at least 2^{128} steps. That is, the speedup by Grover’s search can be easily overcome by doubling the key size)

3.1.3. Remark

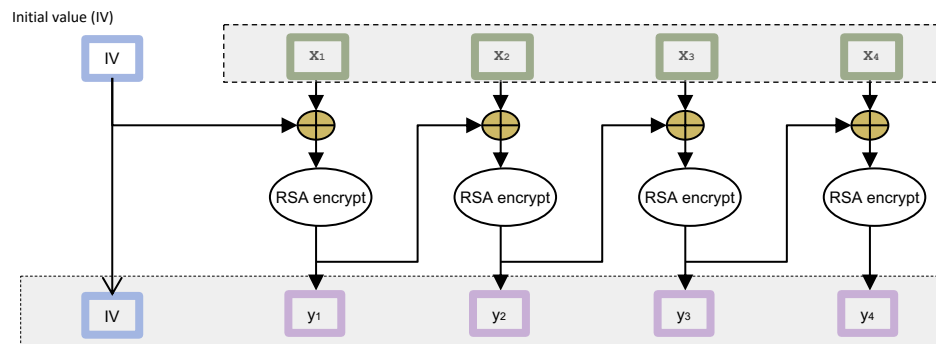
- Efficiency Compares to Symmetric key
- Other PKC schemes

Pitfall: using RSA in the symmetric key setting.

I encountered this quite a few times in Final Year Project evaluations. To encrypt under symmetric setting, students deployed classroom RSA instead of AES. Do not do that, for reason in efficiency and security.

E.g.

- *Bob needs to write an application that provides end-to-end encryption between a digital camera and a mobile phone*
- *Bob feels that RSA is cool. Instead of employing AES, Bob employs classroom RSA as the encryption scheme. The pair of public/private key is treated as the symmetric key. Bob claims that RSA is more secure than AES: it can be proved to be as difficult as factorization, which is believed to be hard.*
- *To encrypt a large image (say 1MB), Bob divides the file into blocks of 256 bytes, and apply RSA on each block using CBC mode.*



Efficiency

- Following NIST recommendation, 128-bit AES and 3072-bit RSA has the equivalent key strength. Larger key might lead to difficulty in key management.
- RSA encryption/decryption is significantly slower than AES.
- See Crypto++ Benchmarks (crypto++ is an C++ library)
<https://www.cryptopp.com/benchmarks.html>

Algorithm	MiB/Second	Cycles Per Byte	Microseconds to Setup Key and IV	Cycles to Setup Key and IV
AES/GCM (2K tables)	102	17.2	2.946	5391
AES/GCM (64K tables)	108	16.1	11.546	21130
AES/CCM	61	28.6	0.888	1625
AES/EAX	61	28.8	1.757	3216
AES/CTR (128-bit key)	139	12.6	0.698	1277
AES/CTR (192-bit key)	113	15.4	0.707	1293
AES/CTR (256-bit key)	96	18.2	0.756	1383

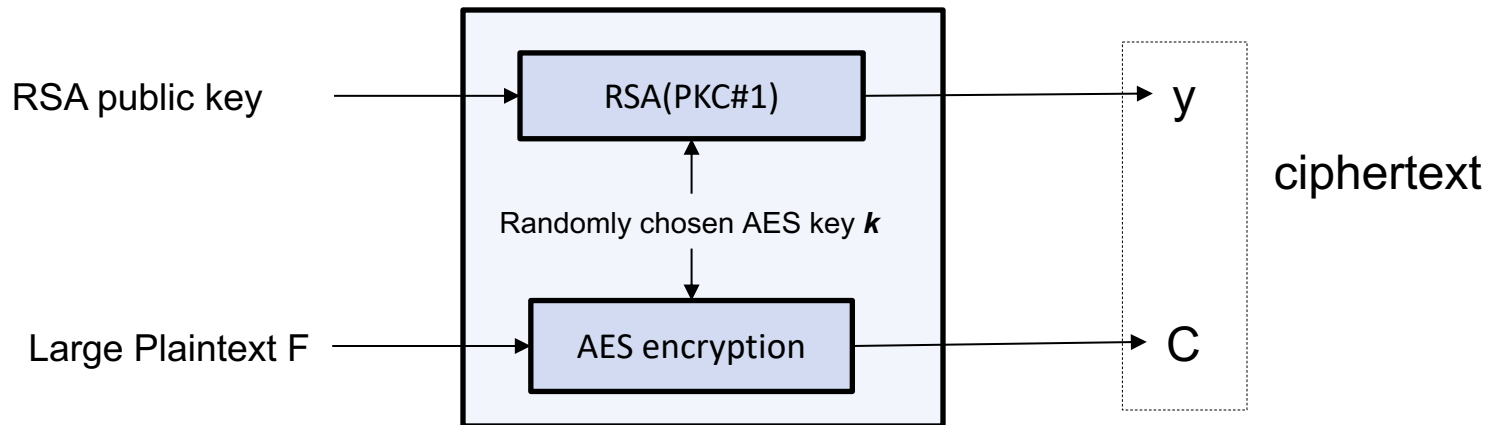
In different order of magnitude.

Operation	Milliseconds/Operation	Megacycles/Operation
RSA 1024 Encryption	0.08	0.14
RSA 1024 Decryption	1.46	2.68

RSA 2048 Encryption	0.16	0.29
RSA 2048 Decryption	6.08	11.12

Using PKC for encryption of large plaintext

- PKC is useful in **authentication**, which is not “encryption”.
- If we want to encrypt a large file F , say a 7GB movie, under Public Key setting, it would be very slow apply RSA to the 7GB movie. The encryption can be efficiently carried out in this way: choose a random AES 128-bit key k , encrypt k using PKC, and encrypt F using AES (with suitable mode, say GCM, CBC or CTR) with k as the key.



Other PKC Schemes:

Discrete log-based PKC

- ElGamal Encryption

ElGamal is a “Discrete Log-based” encryption, whereas RSA is “factorization-based”. There are many choices of Algebraic groups for discrete log-based encryption, e.g. Elliptic Curve. Those using Elliptic Curve are often called Elliptic Curve Cryptography (ECC). Certain choices of ECC reduce the key size. E.g. ~300 bit for equivalent of 2048-bit RSA.

- Paillier Encryption

Paillier Encryption is also discrete-log based.

ElGamal can be easily modified so that it is homomorphic w.r.t. multiplication, whereas Paillier is homomorphic w.r.t. addition.

- If an algorithm can efficiently solve the Discrete Log problem, then the algorithm can easily break the discrete-log based encryption such as Paillier and ElGamal. (In this module, we omit detail of the discrete log problem.)

Lattice-based scheme:

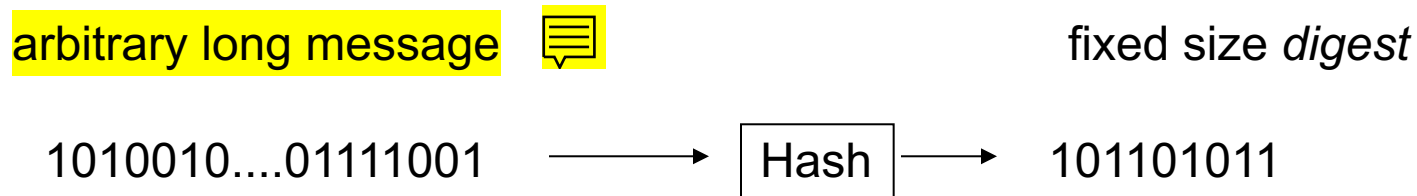
Recap on quantum computer. Currently there are active research in getting Lattice-based PKC. (details omitted).

3.2. Data Authenticity (digest), unkeyed

Remarks: We follow Kerckhoff's principle.
Adversary knows all the algorithms.

Hash (no secret involved!! Adversary knows the algo!!)

A (cryptographic) hash is a function that takes an arbitrary large message as input, and outputs a fixed size (say 160 bits) **digest**.



Security requirement (collision):

- It is difficult for an attacker to find two different messages m_1 , m_2 that “hash” to the same digest. That is,

$$h(m_1) = h(m_2)$$

This is known as **collision-resistant**.

- A hash that is collision-resistant is also **one-way**, that is, given a digest d , it is difficult to find a message m s.t. $h(m)=d$.

Example of hash algorithms that are not secure (collision resistant)

- Taking selected bits from the data.
- CRC checksum.

See https://en.wikipedia.org/wiki/Cyclic_redundancy_check

Popular Hash

- SHA-0, SHA-1, SHA-2, SHA-3
- SHA-0 was published by NIST in 1993. It produces a 160-bits digest. It was withdrawn shortly after publication and superseded by the revised version SHA-1 in 1995.
- SHA-1 is a popular standard. It produces 160-bits message digest. It is employed in SSL, SSH, etc.
- In 1998, an attack that finds collision of SHA-0 in 2^{61} operations was discovered. (Using the straight forward birthday attack, collision can be found in $2^{160/2} = 2^{80}$ operations). In 2004, a collision was found, using 80,000 CPU hours. In 2005, Wang Xiaoyun et al. (Shandong University) gave attack that can finds collision in 2^{39} operations.
- In 2001, NIST published SHA-224, SHA-256, SHA-384, SHA-512, collectively known as SHA-2. The number in the name indicates the digest length. No known attack on full SHA-2 but there are known attacks on “partial” SHA-2, for e.g. attack on a 41-rounds SHA-256 (the full SHA-256 takes 64 rounds)
- In 2005, Xiaoyun Wang et al gave a method of finding collision of SHA-1 using 2^{69} operations, which was later improved to 2^{63} . A collision was found in 2017. It took 110 GPU years, completed 2^{63} SHA1 operations. <https://security.googleblog.com/2017/02/announcing-first-sha1-collision.html>
- In Nov 2007, NIST called for proposal of SHA-3. In Oct 2012, NIST announced the winner, Keccak (pronounced “catch-ack”).

Recent Successful Collision Attacks on **SHA-1** (Feb 2017)

SHAttered
(shattered.io)
Feb 23, 2017



Announcing the first SHA1 collision

February 23, 2017

Posted by Marc Stevens (CWI Amsterdam), Elie Bursztein (Google), Pierre Karpman (CWI Amsterdam), Ange Albertini (Google), Yarik Markov (Google), Alex Petit Bianco (Google), Clement Baisse (Google)

Cryptographic hash functions like SHA-1 are a cryptographer's swiss army knife. You'll find that hashes play a role in browser security, managing code repositories, or even just detecting duplicate files in storage. Hash functions compress large amounts of data into a small message digest. As a cryptographic requirement for wide-spread use, finding two messages that lead to the same digest should be computationally infeasible. Over time however, this requirement can fail due to [attacks on the mathematical underpinnings](#) of hash functions or to increases in computational power.

Today, more than 20 years after of SHA-1 was first introduced, we are announcing the first practical technique for generating a collision. This represents the culmination of two years of research that sprung from a collaboration between the [CWI Institute in Amsterdam](#) and Google. We've summarized how we went about generating a collision below. As a proof of the attack, we are [releasing two PDFs](#) that have identical SHA-1

Recent Successful Collision Attacks on SHA-1 (Feb 2017)

- Done by a team from CWI and Google
- Two PDF files with the same hash values as attack proof:
 - <https://shattered.io/static/shattered-1.pdf>
 - <https://shattered.io/static/shattered-2.pdf>
- Defense mechanisms:
 - **Use SHA-256 or SHA-3 as replacement**
 - Visit shattered.io to test your PDF

Read

<https://shattered.io/>

The screenshot displays the SHattered website, which announces the first concrete collision attack against SHA-1. The website has two versions: a blue one and a red one. Both versions feature the CWI and Google logos and list the names of the researchers: Marc Stevens, Elie Bursztein, Ange Albertini, and Yarik Markov. Below the website, a terminal window shows the command `sha1sum *.pdf` being executed, resulting in two PDF files with identical SHA-1 hashes: `88762cf7f55934b34d179ae6a4c80cadccbb7f0a 1.pdf` and `88762cf7f55934b34d179ae6a4c80cadccbb7f0a 2.pdf`. The terminal also shows the command `sha256sum *.pdf` being executed, resulting in two PDF files with identical SHA-256 hashes: `2bb787a73e37352f92383abe7e2902936d1059ad9f1ba6daaa9c1e58ee6970d0 1.pdf` and `44488775d29bdef7993367d541064dbdda50d383f89f0aa13a6ff2e0894ba5ff 2.pdf`. A progress bar at the bottom of the terminal indicates that the process is 0.64G in size and has been running for 8-11h.

- MD5
- Designed by Rivest. MD, MD2, MD3, MD4, MD5, MD6.
- MD6 was submitted to NIST SHA-3 competition, but did not advance to the second round of the competition.
- MD5 was widely used. It produces 128-bit digest.
- In 1996, Dobbertin announced a collision of the compress function of MD5.
- In 2004, collision was announced by Xiaoyu Wang et al. The attack was reported to take one hour.
- In 2006, Klima give an algorithm that can find collision within one minute on a single notebook.

An application scenario of unkeyed Hash (without secret keys)

Application of (unkeyed) Hash for integrity

Consider this example

- Alice downloaded a software `vlc-2.2.8-win32.exe` from the web. Is the downloaded file authentic?

(see the “checksum” in next slide)

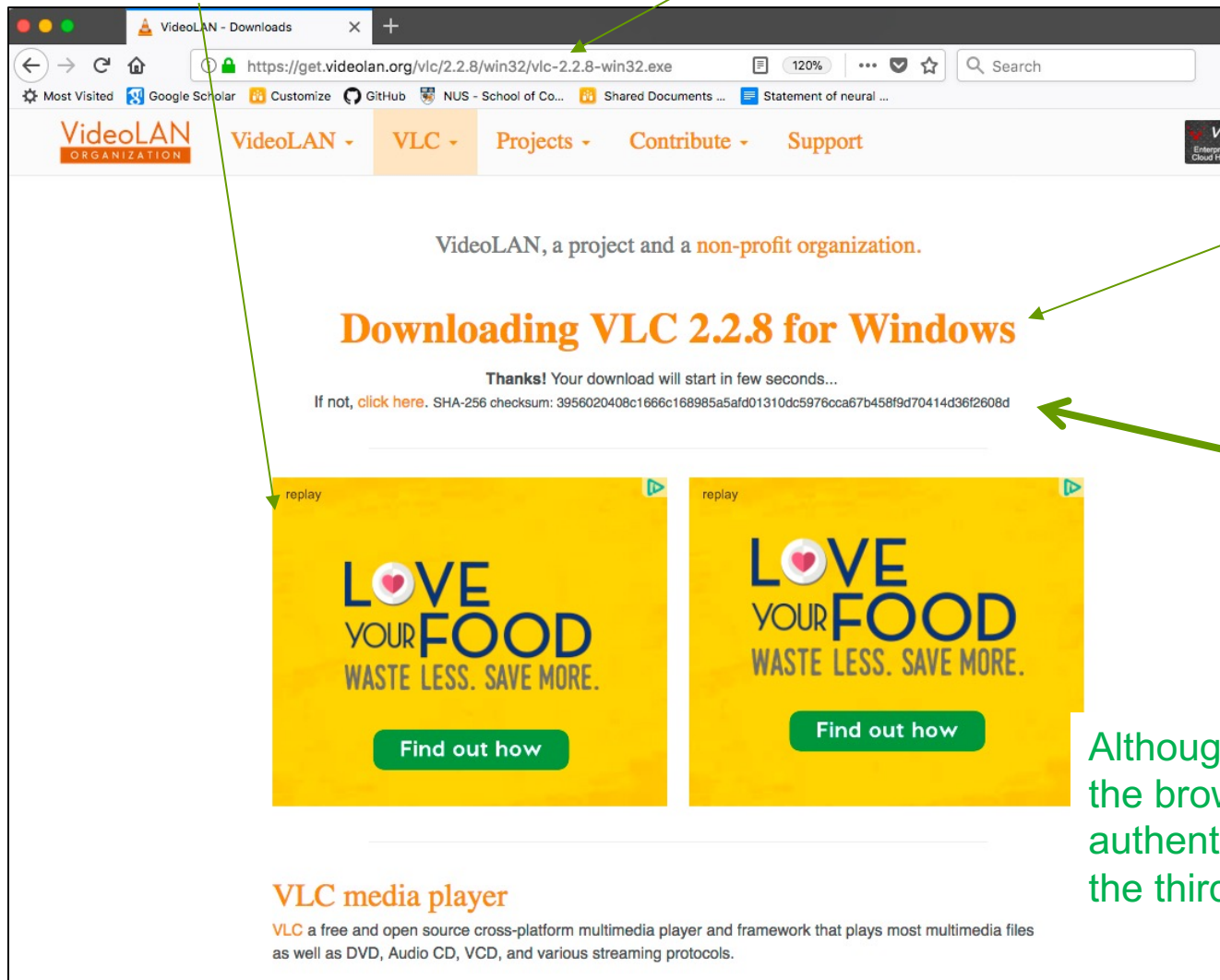
Specifically,

1. Alice visits the website of VLC.
2. Since the website is hosted with HTTPS protocol, Alice is being assured that the content displayed on the browser is from VLC and authentic. We will discuss https later and right now, let's treat all info displayed by browser as authentic.
3. However, the downloading site is a 3rd-party, i.e. the actual file `vlc-2.2.8-win32.exe` is hosted in another website. The communication channel to the 3rd party website is not secure. There is also a possibility that the 3rd party website is malicious and giving out virus infested software.
4. To verify that the file indeed is the original, after Alice downloaded the file, she can check the integrity of the file by matching the “hash” of the file with the “SHA-256 checksum” displayed in the browser. If they match, then Alice is very sure that the file is intact. If not, certainly the file is corrupted.

Ads from 3rd party.

(might not via https and videolan.org is unable to control what being shown)

https. So the content displayed here is from **videolan.org** and is authentic.



Actual file is hosted in a third party site

the digest
(aka checksum).

Although the information displayed in the browser is verified to be authentic, what about the file from the third party site?

Question: why "videolan.org" in the url is boldfaced? (to be discussed later)

Application of (unkeyed) Hash for integrity

In this scenario, we assume that there is a secure channel to send short piece of information. (in VLC example, this channel is the https)

Let F be the original data. Alice obtains the digest $h(F)$ from the secure channel.

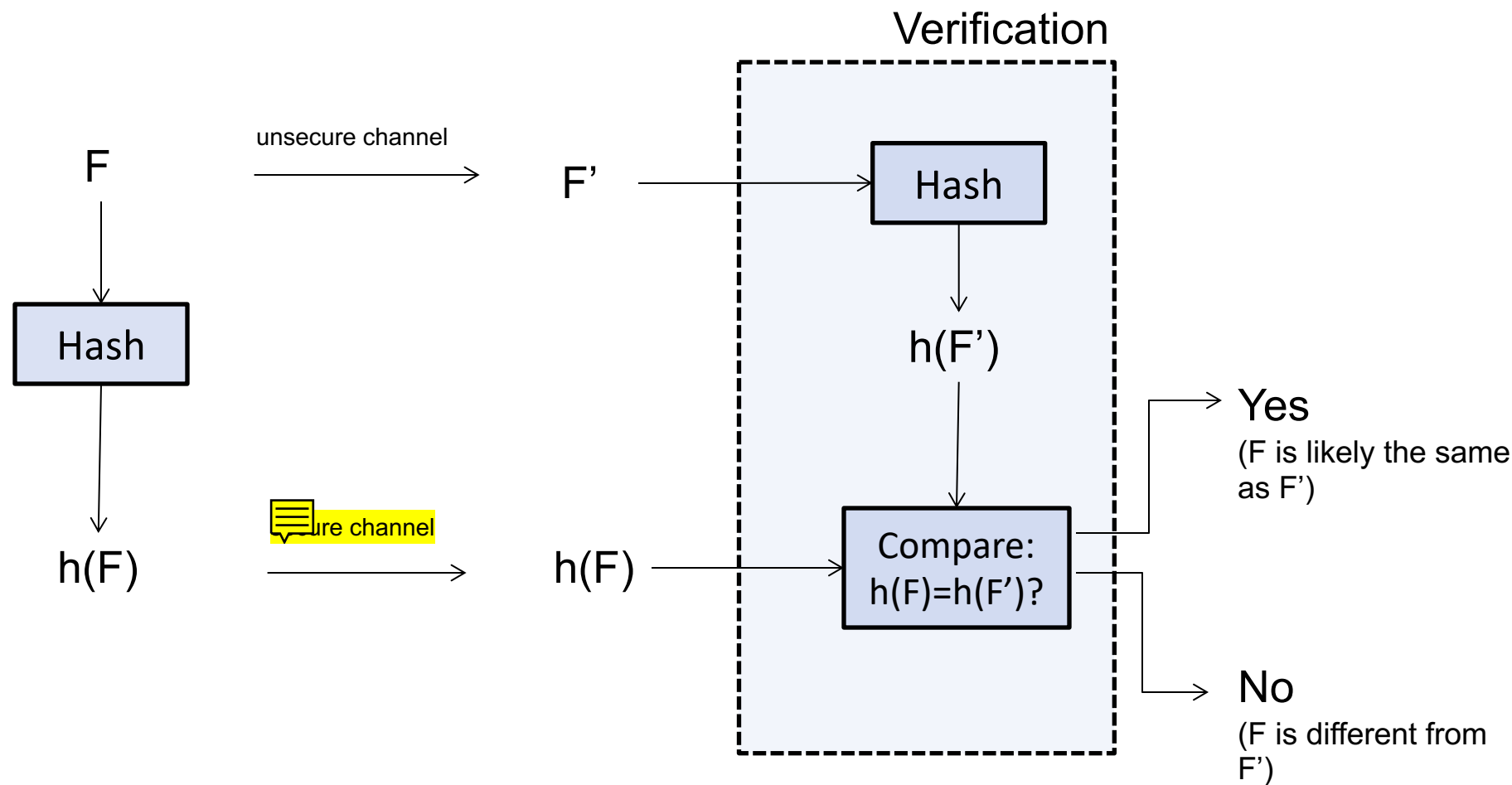
Alice obtains a file, say F' , whose origin claims that the file is F . Alice computes and compares the digests $h(F)$, $h(F')$.

If they are the same, F' is indeed same as F with very high confidence. If they are different, then F' must be different from F , i.e. integrity compromised.

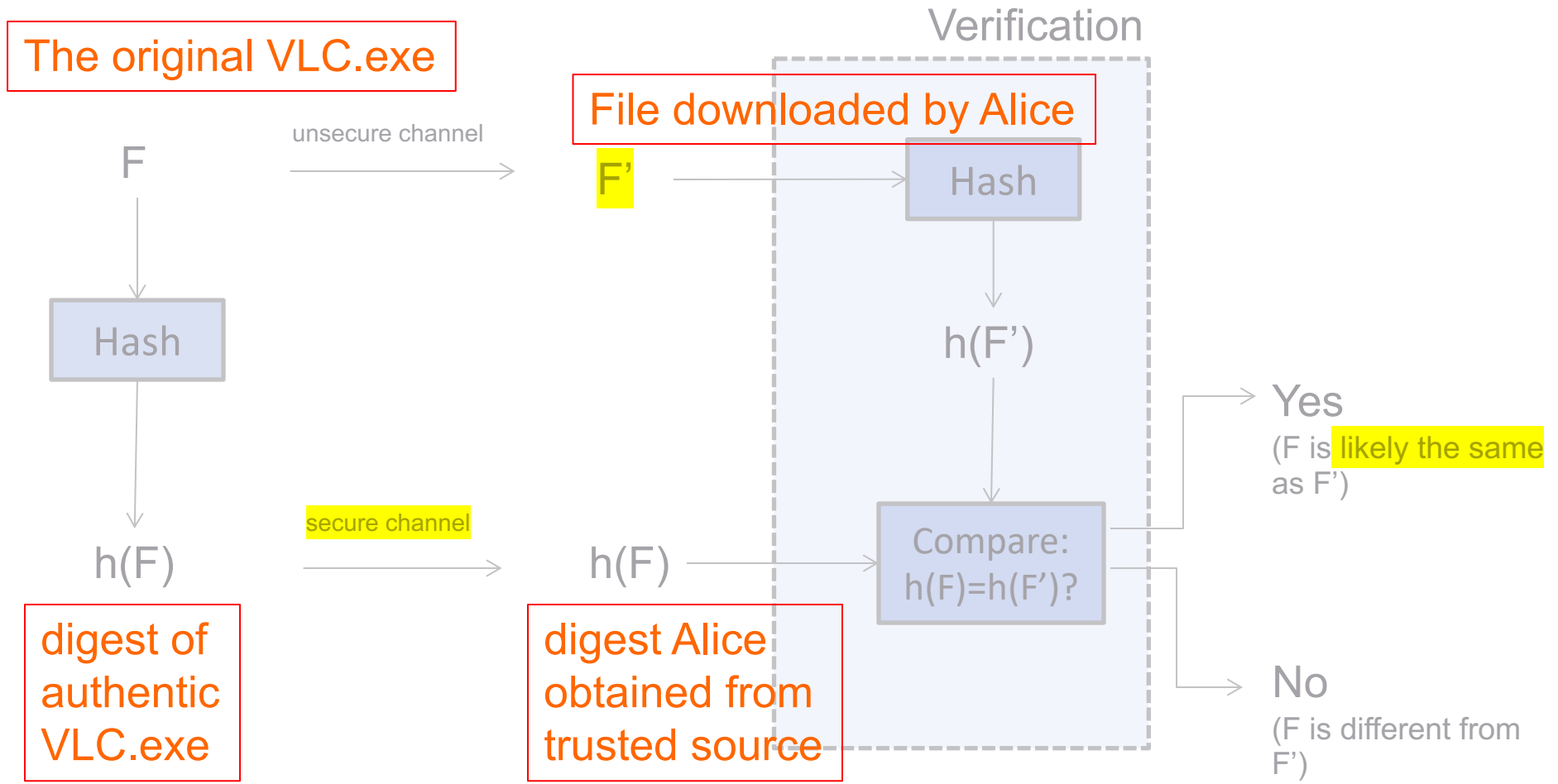
$$h(F) = h(F') \quad \Rightarrow \quad \text{with high probability, } F=F'$$

$$h(F) \neq h(F') \quad \Rightarrow \quad F \neq F'$$

(unkeyed) hash



(unkeyed) hash



Remarks

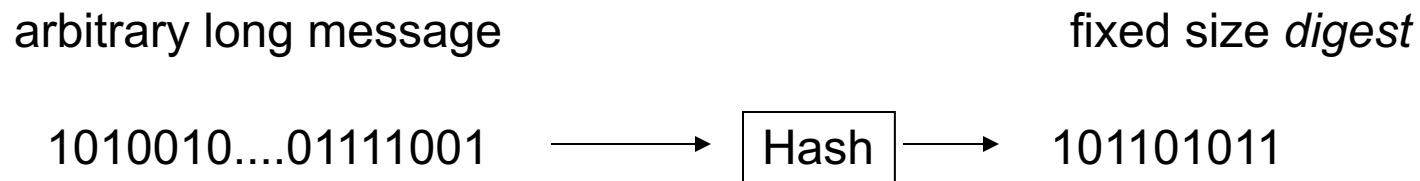
- What would an attacker do? (*find a collision*)
 - attacker's goal: makes Alice accept a malicious file;
 - attacker's capability: (1) knows the file VLC.exe, (2) can send any file F' across)



- We may argue that with the digest, the verifier can be assured that the data are authentic, and thus authenticity of the data origin is achieved. Nevertheless, in many literatures, when there is no “secret key” involved, we refer to the problem as an “integrity” issue.

3.3 Data Origin Authenticity (mac), Keyed

Recap: Hash (no secret involved)



Security requirement (collision):

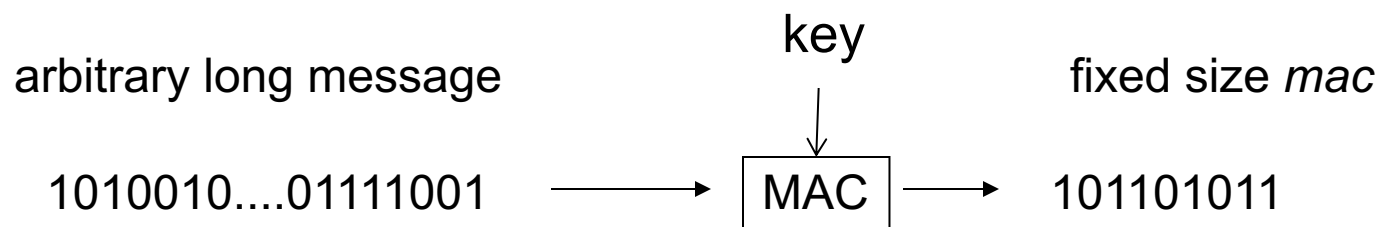
- **(Collision-resistant)** It is difficult for an attacker to find two different messages m_1 , m_2 that “hash” to the same digest. That is,

$$h(m_1) = h(m_2)$$

Note: collision-resistant implies one-way

Keyed-Hash (aka MAC) (a secret key is involved)

A keyed-hash is a function that takes an arbitrary large message and a secret key as input, and outputs a fixed size (say 160 bits) *mac* (*message authentication code*).



- **Security requirement (forgery):** After seen multiple valid pairs of messages and their corresponding mac, it is difficult for the attacker to forge the mac of a message not seen before. (The precise formulation is quite involved, and with many variants. Higher level crypto module CS4236 would cover this)

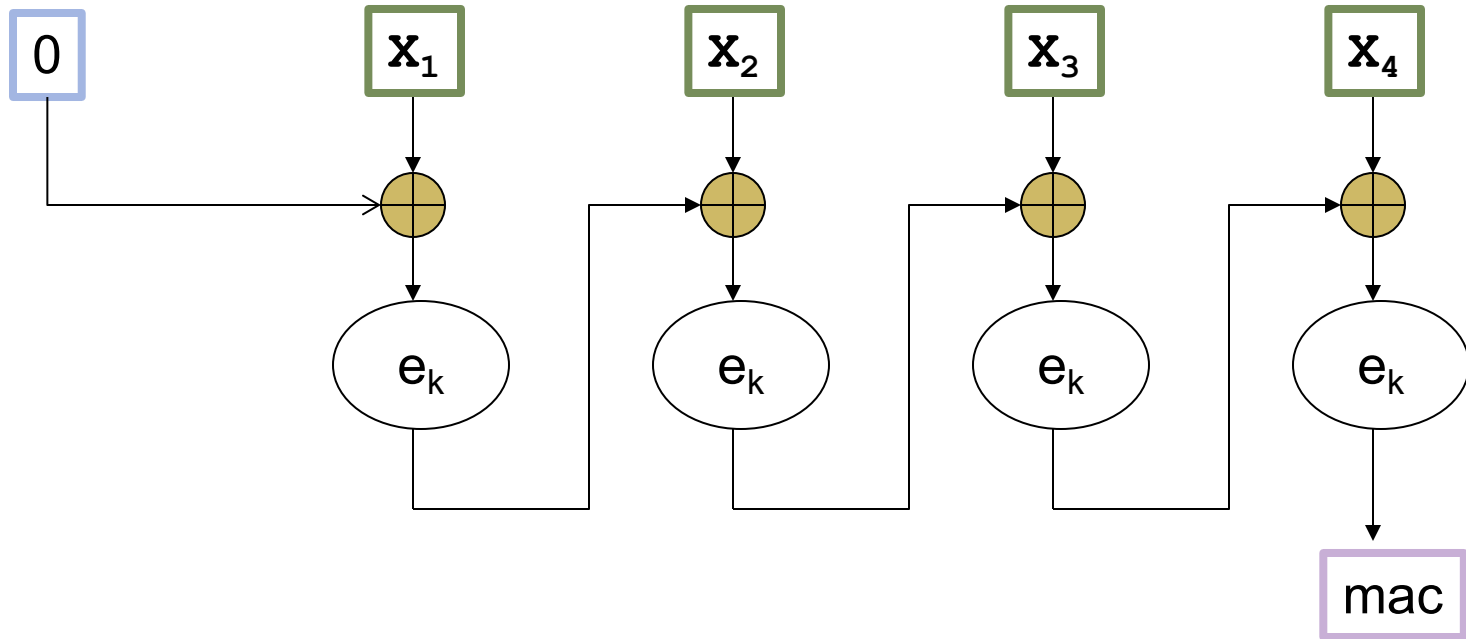
Popular keyed-hash (MAC)

- CBC-MAC (based on AES operated under CBC mode)
- HMAC (based on SHA)
Hashed-based MAC

standard: RFC 2104. <http://tools.ietf.org/html/rfc2104>

CBC-mac

Initial value (IV)



$$\text{HMAC}_k(x) = \text{SHA-1}((K \oplus \text{opad}) || \text{SHA-1}((K \oplus \text{ipad}) || x))$$

where

opad = 3636...36 (outer pad)

ipad = 5c5c...5c (inner pad)

(the above are in hexadecimal)

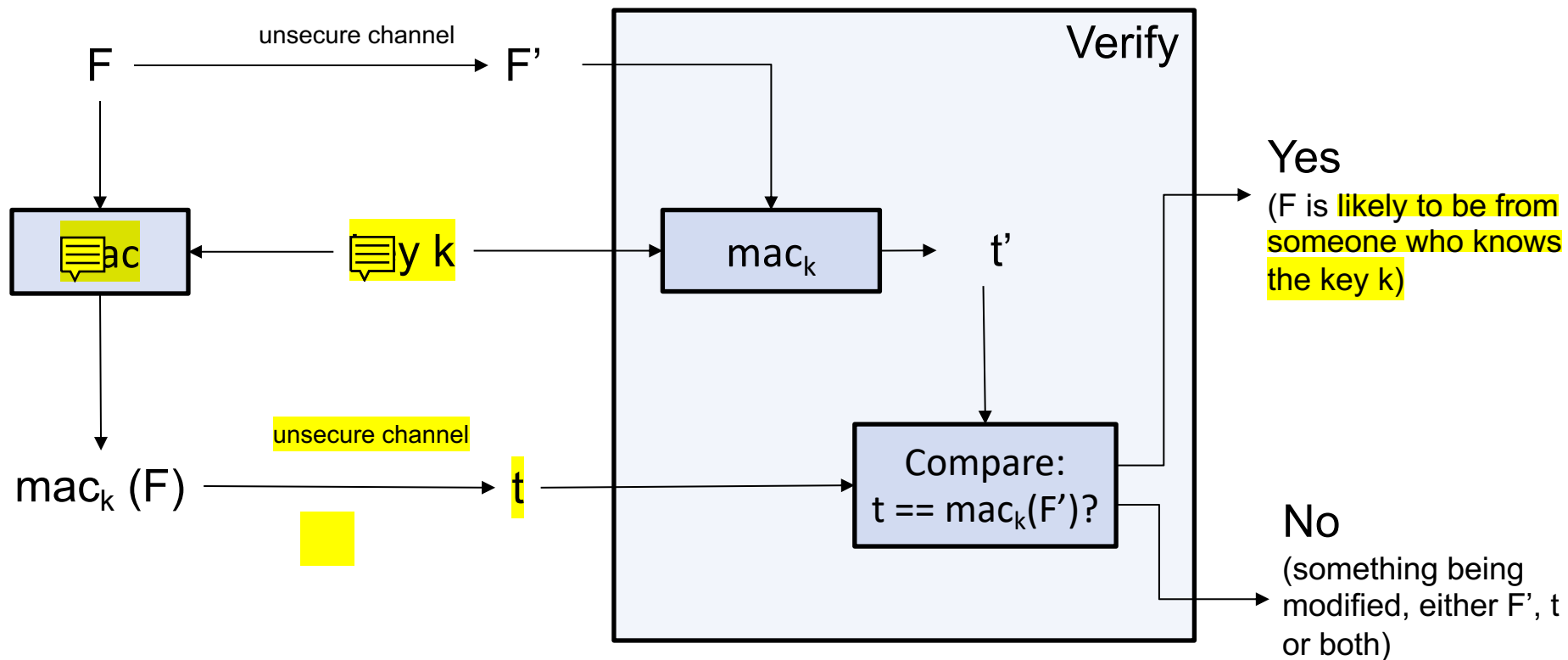
An application scenario for mac

- In the previous example (on vlc), we assume that there is a secure channel to send the digest.
- There are scenarios that we don't have a secure channel to deliver the digest. (in the vlc example, it relies on the fact that we have https. What if we don't have https?)
- In such scenarios, we can protect the digest with the help of some secrets.
 - In the symmetric key setting, it is called the **mac**.
 - In the public key setting, it is called the **digital signature**.

mac (Message Authentication Code)



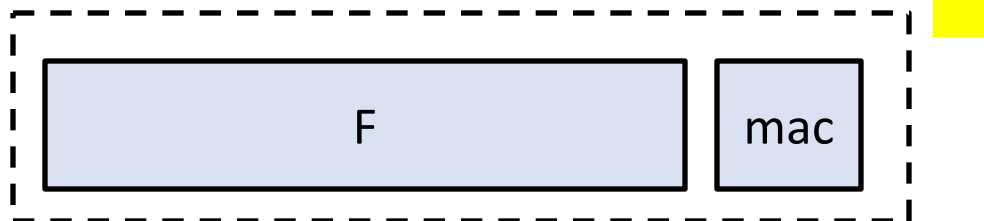
In this setting, the mac might be modified by attacker. If such case happened, it can be detected with high probability.



Security Requirement: Without knowing the key k , it is difficult to forge a mac.

Remark

- What would an attacker do? (*forged a valid pair of (message, mac)*)
- Note that there is no issue on confidentiality. In fact, the data F can be sent in clear.
- Typically, the *mac* is appended to F. They are then stored as a single file, or transmitted through the communication channel together. (Hence, *mac* is also called the *authentication tag*, or *authentication code*)
- Later, an entity who wants to verify the *authenticity* of F, can carry out the verification process using the secret key.

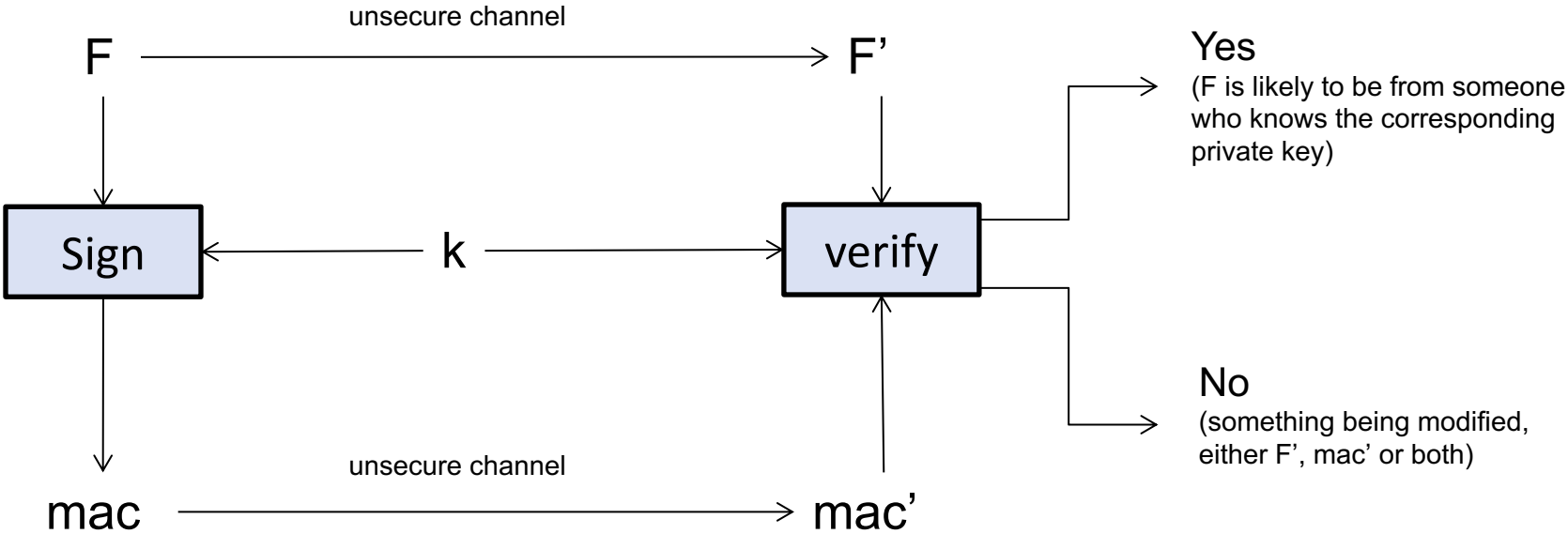


3.4 Data Origin Authenticity (Signature), Asymmetric key

Signature

- The public key version of MAC is called Signature.
- Here, the owner uses the ***private key*** to generate the signature. The public can use the ***public key*** to verify the signature.
- So, anyone can verify the authenticity of the data, but only *the person who know the private key* can generate the signature.

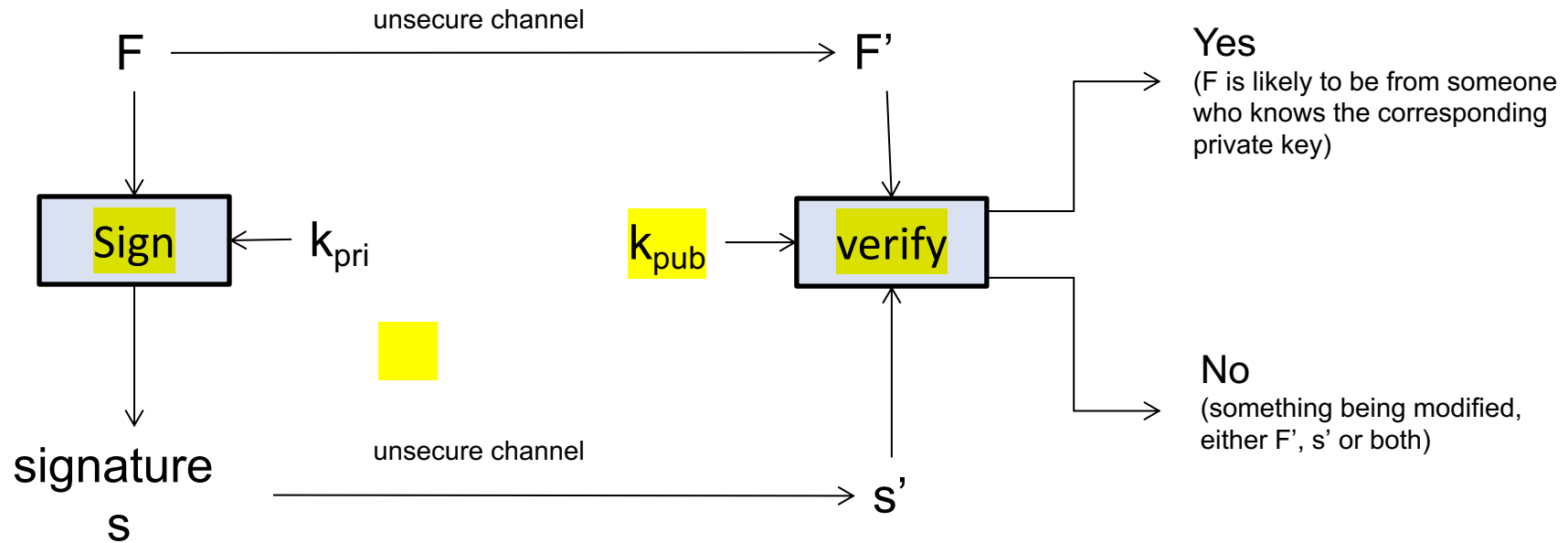
mac



Signature

Verifier and Signer using different key.

k_{pri} : private key
 k_{pub} : public key



Security Requirement: Without knowing the private k_{pri} , it is difficult to forge a signature.

Remark

- Likewise, the computed signature is typically appended to F and stored as a single file.
- Note that the signature is computed using the private key and F .



- When we say that
“Alice signs the file F ”,
we mean that Alice computes the signature s , and then appends it to F .
- Later, the authenticity of F can be verified by anyone who knows the public key. The valid signature ***can only be*** computed by someone who knows the private key. So, if it is valid, then F must be authentic.
- Anyone can verify using the public key.

What so special of signature compare to mac?

- Beside the ease of key management compare to mac, signature achieve an additional security requirement.
- We can view the digital signature as the counterpart of the handwritten signature in legal document. A legal document is authentic or certified, if it has the correct handwritten signature. No one, except the authentic signer, can generate the signature.
- Signature scheme achieves **Non-repudiation**.

Non-Repudiation: Assurance that someone cannot deny previous commitments or actions.

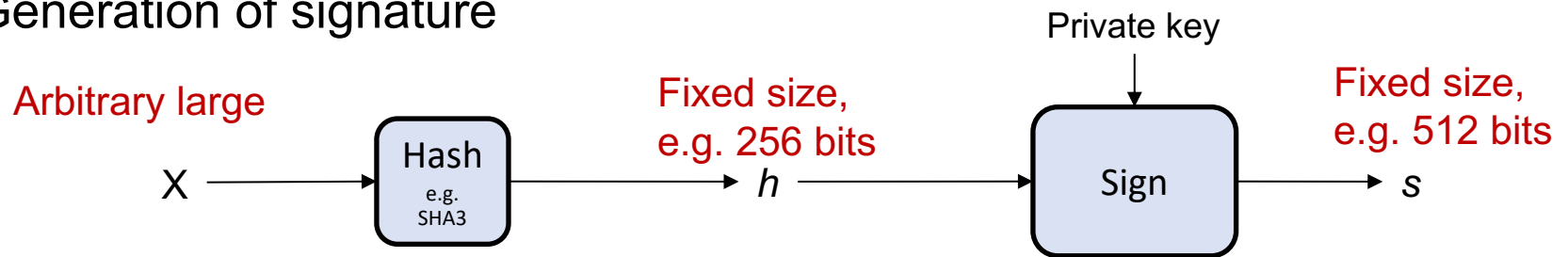
Example on non-repudiation

- (using mac). Suppose Alice sent Bob a message appended with a mac. The mac is computed with a key shared by Alice and Bob. Later Alice denied that she had sent the message. When confronted by Bob, Alice claimed that Bob generated the mac.
- (using signature). Suppose Alice sent Bob a message, signed using her private key. Later, she wanted to deny that she had sent the message. However, she was unable to so. This is because only the person who knows the private key can sign the message and only Alice knows the private key. Hence, the signature is a proof that Alice generated the message.

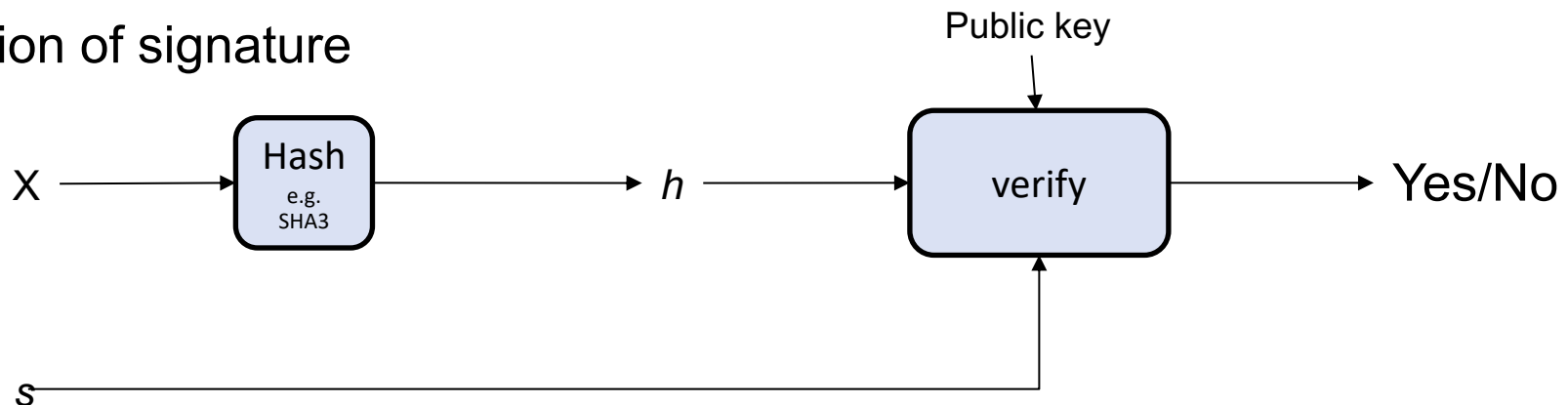
Design of Signature scheme

- Most signature scheme consists of two components. An unkeyed hash, and the sign/verify algorithm.

Generation of signature



Verification of signature



Popular Signature scheme

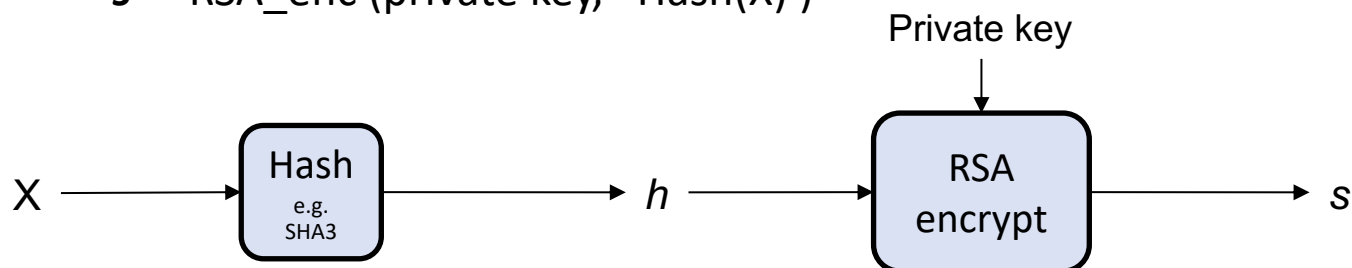
- A popular group of schemes use RSA for the sign/verify component. :
RSASSA-PSS, RSASSA-PKCS1: signature scheme based on RSA
- DSA (Digital Signature Algorithm) is another popular standard whose security depends on discrete log.

https://en.wikipedia.org/wiki/Digital_Signature_Algorithm#Implementations

RSA-based signature

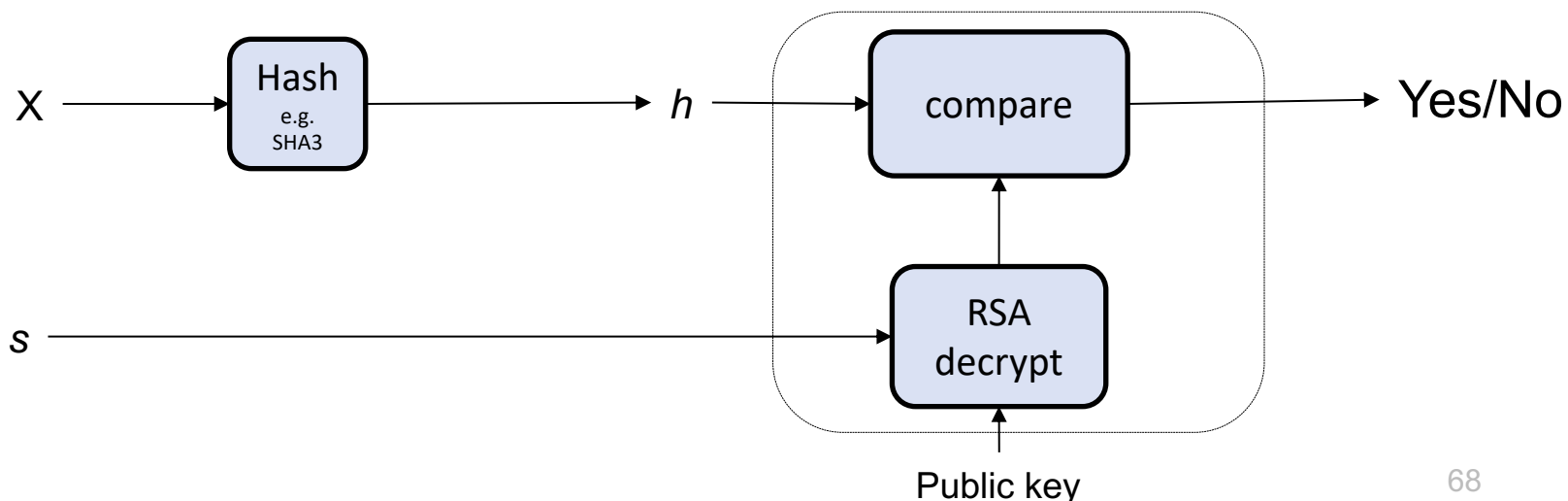
- Use RSA for signing and verification. Essentially, the signature is the “encrypted” digest. *A messy notation issue: Previously, we use public key to encrypt. Here, we use private key to encrypt. Recall that for RSA, we can flip the role.*

$$s = \text{RSA_enc}(\text{private key}, \text{Hash}(X))$$



- Verification of (X,s) is done by using the public key.

If $\text{Hash}(X) = \text{RSA_dec}(\text{public key}, s)$ then accept, else reject.



Not necessary to have “encryption” in signature scheme

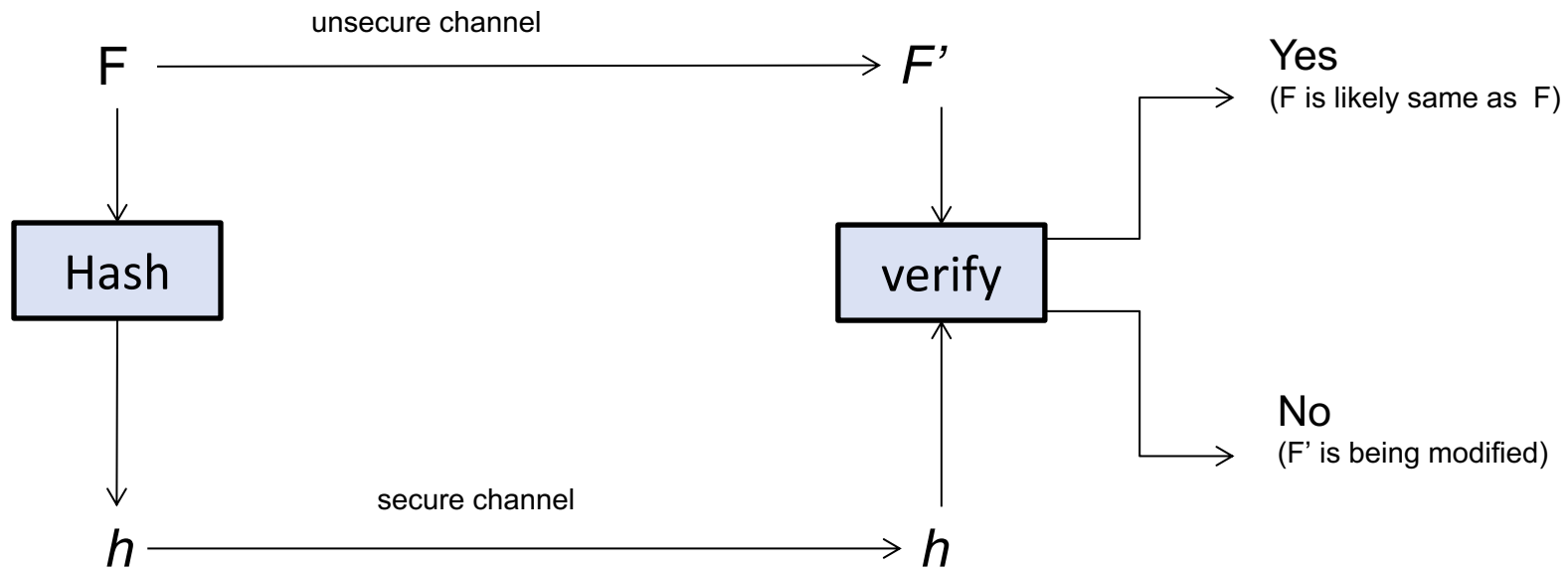
- Many books/documents describe a signature as the encryption of the hash. This is one method. Not all signature scheme incorporate “encryption”.
- Popular schemes such as DSA employ do not “encrypt” (there isn’t a way to decrypt).
- So, a more accurate way is to say: “hash-and-sign”

The method of hash-and-encrypt is a special case of hash-and-sign.

Summary

3.3 Digest (Hash)

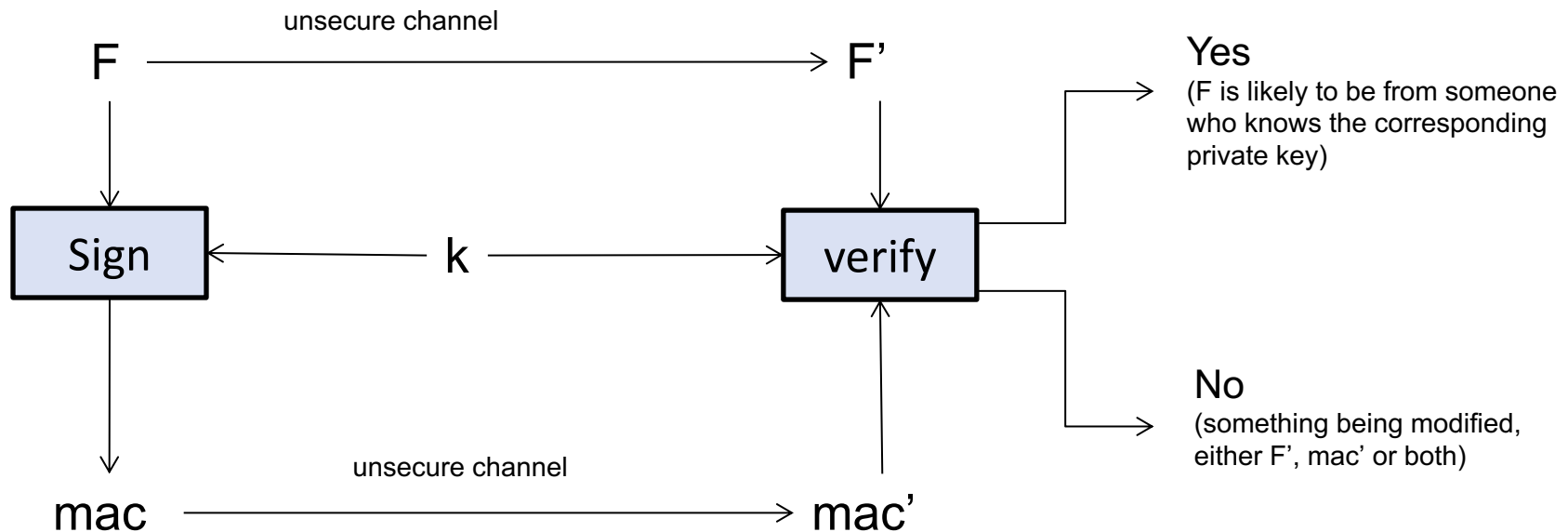
The digest must be sent through secure channel



Security Requirement: Different to find a F' with the same digest

3.4 MAC (Message Authentication Code)

The mac can be sent through an unsecure channel. Both

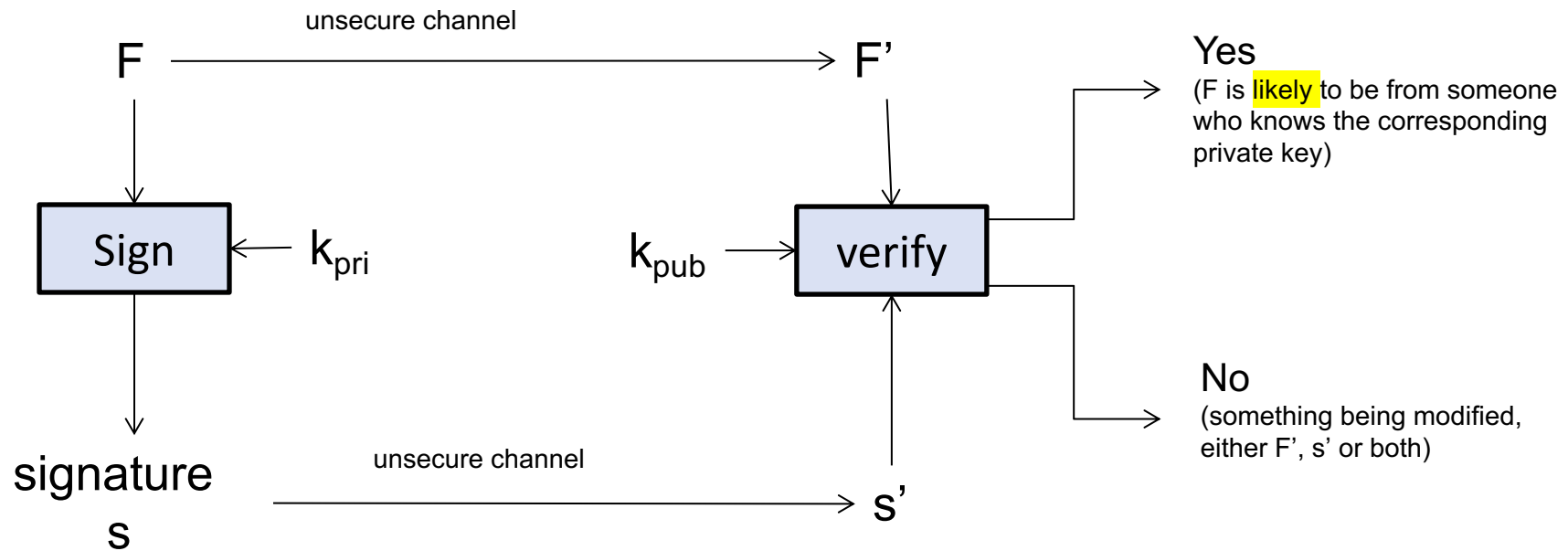


Security Requirement: Without knowing k , it is difficult to forge a mac.

3.5 Signature

Verifier and Signer using different key.

k_{pri} : private key
 k_{pub} : public key



Security Requirement: Without knowing the private k_{pri} , it is difficult to forge a signature.

3.5. Some attacks and pitfalls



3.5.1 Birthday attacks

- Hashes are designed to make collision difficult to find. Recall that a collision consists of two different messages x_1, x_2 that give the same digest, i.e.

$$h(x_1) = h(x_2) \quad \text{and} \quad x_1 \neq x_2$$

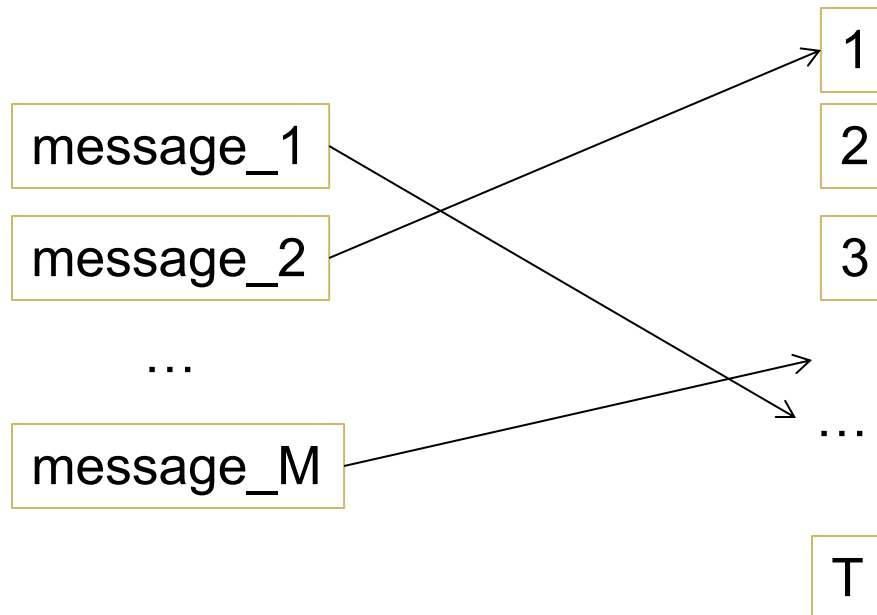
- Nevertheless, all hash functions are subjected to **birthday attack**. (similar to exhaustive search on encryption scheme).



- In a class of 25 students, with probability more than 0.5, there is a pair of students having the same birthday.
- Suppose we have M messages, and each message is tagged with a value randomly chosen from $\{1, 2, 3, \dots, T\}$. If

$$M > 1.17 \sqrt{T}$$

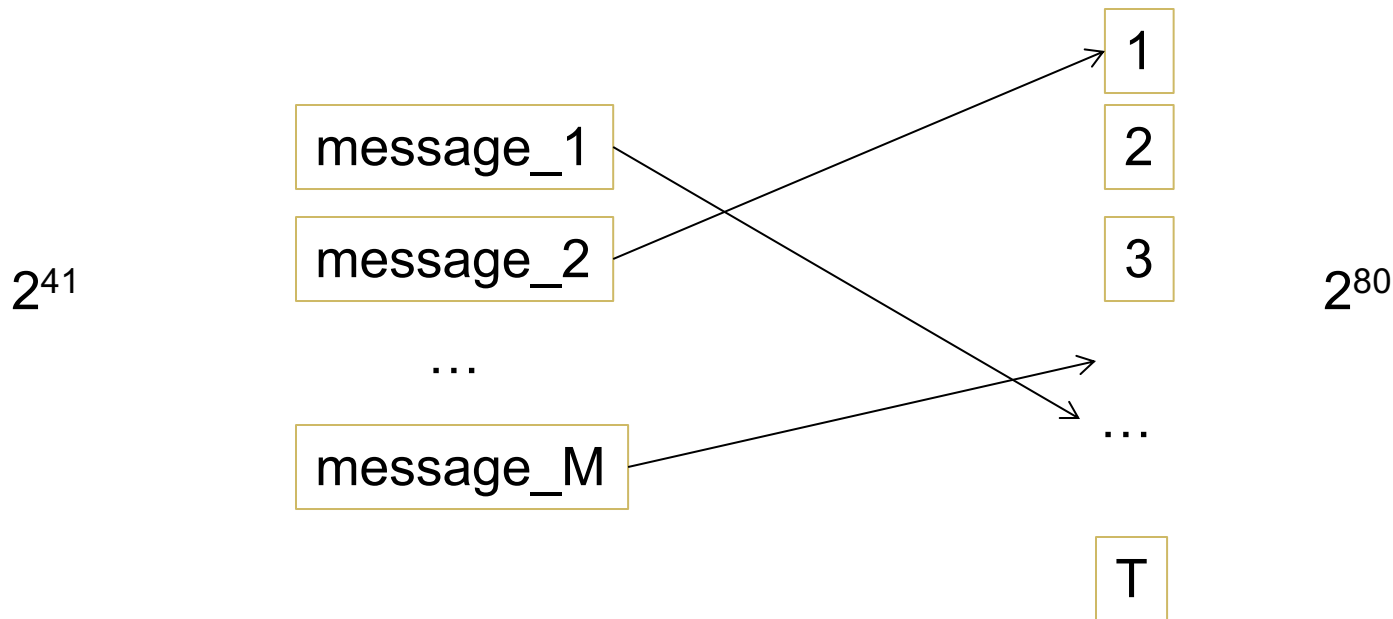
then, with probability more than 0.5, there is a pair of messages tagged with the same value.



a lot of T 's
but few messages

- Suppose the digest of a hash is 80 bits ($T = 2^{80}$). An attacker wants to find a collision.

If the attacker randomly generates 2^{41} messages ($M = 2^{41}$), then $M > 1.17 T^{0.5}$. Hence, with probability more than 0.5, among the 2^{41} messages, two of them give the same digest.



in general, the probability is approximately $1 - \exp(-M^2 / (2T))$

Length of digest

Read the NIST recommendation of key-length
(<http://www.keylength.com/en/4/>)

When key length for symmetric-key is 112, the corresponding recommended length for digest is at least 224. Why the digest length has to be significantly larger?

(birthday attack)

Question

- Back to the example of ECB mode-of-operation. (penguin)
- One remedy is as follow: each block is to be encrypted with an IV (i.e. AES (IV xor plaintext)) Although secure, it increases the ciphertext size. This is because the IV for each block has to be included in the ciphertext. Suppose each IV is 128 bits, there will be an overhead of 128 bits per block. Hence it is desirable to keep the size of IV small.
- What about setting the IV to 16 bits?

Suppose it is decided to use 16 bits for the IV, and each image has around $2^{10} = 1024$ blocks. The IVs are Independently and randomly chosen.

1. What is the probability that the first two blocks have the same IV? 2^{-16}
2. What is the probability that, among the 1024 blocks, there exists two blocks having the same IV? *Extremely high, certainly > 0.5*

3.5.2 Using encryption for the purpose of authentication

Encryption schemes may provide false sense of security. Consider this design of a mobile apps from a company XYZ.

The mobile phone and a server share a secret 256-bit key k . The server can send instructions to the mobile phone via sms. (Note that sms only consist of readable ascii characters. We assume that there is a way to encode binary string using the readable characters). The format of the instruction is:

X P

where X is an 8-bit string specifying the operation, and P is a 120-bit string specifying the parameter. So, an instruction is of size 128 bits. If an operation doesn't need a parameter, P will be ignored. There is a total of 15 valid instructions.

E.g.

00000000 P : send the GPS location to phone number P via sms. If P is not a valid phone number, ignore.

11110000 P : rings for P seconds. If $P > 10$, ignore.

10101010 : self-destruct now!

- An instruction is to be encrypted using AES CBC-mode with 256-bit key, encoded to readable characters and sent as sms. (recap: block size of AES is 128 bits).
- After a mobile phone received a sms, it decrypts it. If the instruction is invalid, it ignores the instruction. Otherwise, it executes the instruction.
- The company XYZ claims that *“256-bit AES provides high level of security, and in fact is classified as Type 1 by NSA. Hence the communication is secure. Even if the attackers have compromised the base station, they are still unable to break the security”*.
- Something is wrong here.
(If an attacker sends a randomly chosen message to the mobile phone, what is the probability that the mobile phone self-destruct?)

Remarks

- Encryption is designed to provide confidentiality. It does not necessary guarantee integrity and authenticity.
- In the previous example, XYZ wants to achieve “authenticity”, but wrongly employing encryption to achieve that. (Some encryption schemes also provide authenticity, but not all. The example unfortunately chose one that doesn't.)
- A secure design could use mac instead of encryption.

[optional] Also note that a lot of details are omitted. Simply adding mac to the instructions is not sufficiently secure. To prevent “replay” attack, “nonce” is required.

3.5.3 Time-space Tradeoff

Reference:

- See http://en.wikipedia.org/wiki/Rainbow_table
Section “Precomputed hash chains”.

The above wiki page describes “Rainbow table” which is an improved variant of time-memory tradeoff. The original basic variant is described in the section “Precomputed hash chain”.

3.5.3 Time-memory tradeoff for Dictionary Attack

- Let consider a hash $H()$ which is collision resistant (e.g. SHA3() is believed to be so). Since it is collision resistant, it is also one-way. Thus, given a digest y , it is difficult to find a x s.t. $H(x)=y$.
- Suppose we know that the message x is chosen from a relatively small set of *dictionary* D . For illustration, let's assume x is a randomly and uniformly chosen 50-bits message.
- Even if $H()$ is one-way, given the digest y , it is still feasible to find a x in D s.t. $H(x)=y$. This can be done by exhaustively searching the 2^{50} messages in D . Although feasible, this would take days of computing time. As the attacker, we want to speedup the inverting process to support “realtime” attack.

Using a large table.

- We are allowed to perform pre-processing. One straightforward method is to build a dataset with 2^{50} elements $(H(x), x)$ for $x = 0, 1, 2, \dots, 2^{50}-1$

(here, we view the 50 bits message as an integer)

and store these elements in a data structure T that supports fast lookup (for instance, hash table that facilitates constant-time lookup)

Now, given a digest y , we can query the data structure and readily find the associated x .

However, such table is too large. (2^{50} entries = 2^{10} “Tera” entries)

The time-memory tradeoff is a technique to “tradeoff” time (i.e. slower lookup time) for memory (i.e. smaller storage)

Time-memory Tradeoff:

- The main idea is to use a precomputed hash-chain. (The term “hash-chain” appears in different context and refer to different techniques.)
- Define a *reduce function* $R()$ that maps a digest y to a word w in the dictionary D . For illustration, if D consists of all 50-bit messages, and each digest is 320 bits, then a possible reduce function simply keeps the first 50 bits of input.

$$R(b_0b_1\dots b_{320}) = b_0b_1\dots b_{49}$$

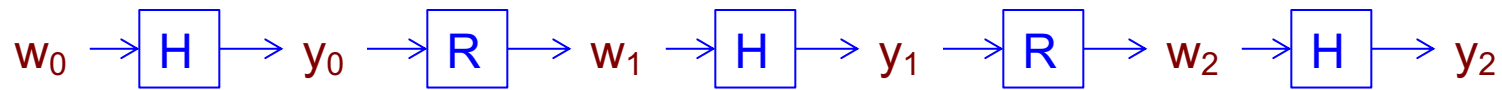
Here is a pre-computed chain. It starts from a randomly chosen word w_0 in D .

$$w_0 \rightarrow y_0 = H(w_0) \rightarrow w_1 = R(y_0) \rightarrow y_1 = H(w_1) \rightarrow w_2 = R(y_1) \rightarrow y_2 = H(w_2)$$

e.g

$$\text{"hello"} \rightarrow \text{A0C0...20} \rightarrow \text{"qwert1"} \rightarrow \text{03F0...50} \rightarrow \text{"Pikachu"} \rightarrow \text{77FF...3A}$$

$$w_0 \rightarrow y_0 = H(w_0) \rightarrow w_1 = R(y_0) \rightarrow y_1 = H(w_1) \rightarrow w_2 = R(y_1) \rightarrow y_2 = H(w_2)$$



Call w_0 the starting-point, and y_2 the ending-point. Store the pair (w_0, y_2) in the data-structure T . The process is repeated with other randomly chosen starting points.

During the query, given a digest y . First search for y in the data-structure T .

- Suppose y is in T . That is, y is one of the ending-point. Let's assume that w_0 is the corresponding starting point (hence $y = y_2$ in the above chain). Note that a pre-image of y is w_2 , but at this point, we don't know the value of w_2 . Nevertheless, the fact that y is the ending-point implies that w_2 is within the chain starting from w_0 . So, we can start to construct the chain $w_0, y_0, w_1, y_1, w_2, y_2$. When the process hits y_2 , we have found the w_2 .
- Suppose y is not in T . Compute $y' = H(R(y))$. Search the data-structure for y' . Suppose y' is in T . Let's assume that the starting-point be w_0 (hence $y' = y_2$). With high chances, $y = y_1$. So, a pre-image of y is w_1 (i.e. $H(w_1) = y$). At this point, we don't know w_1 . But by constructing the chain from w_0 , the pre-image w_1 can be found.

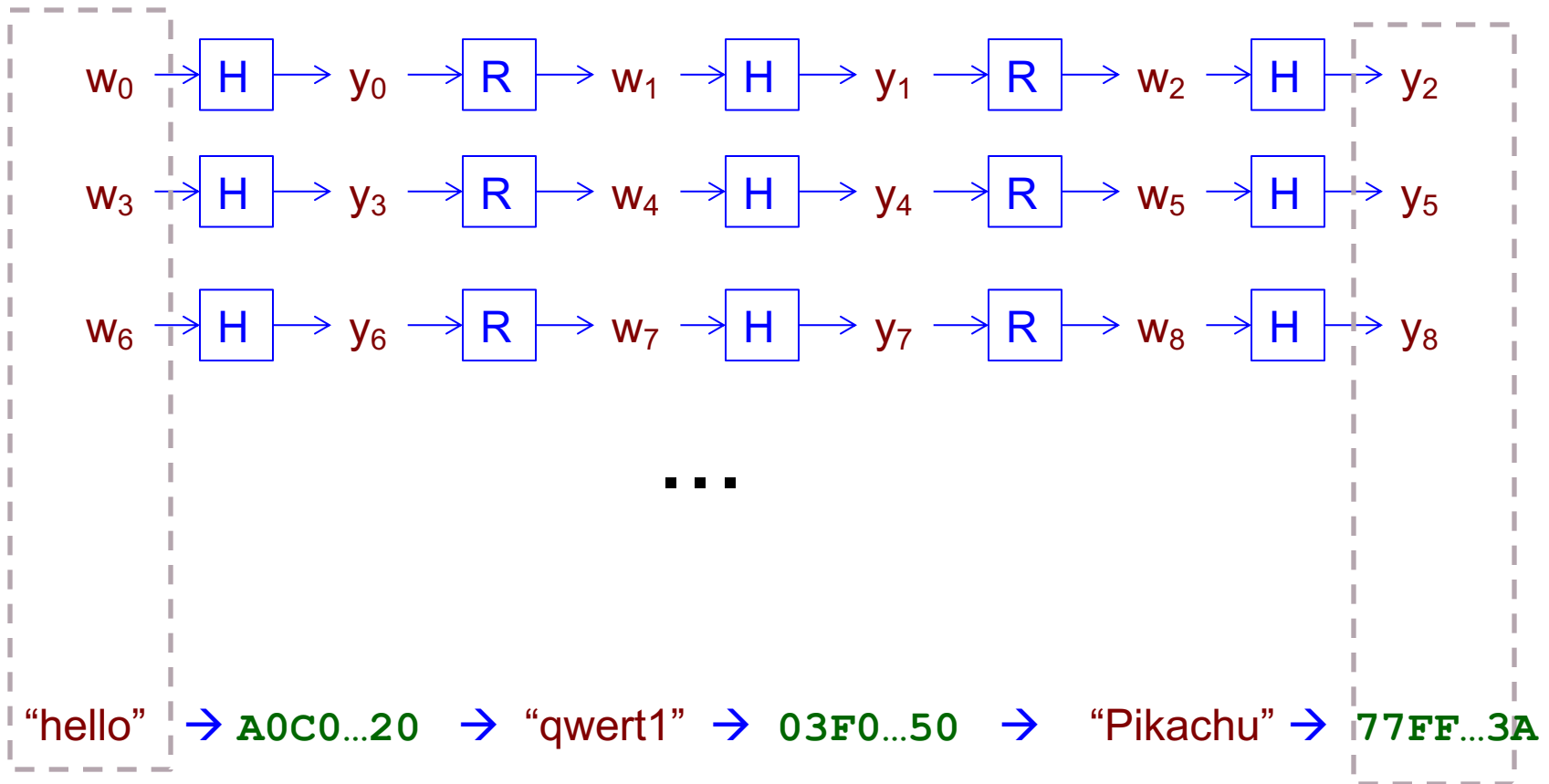
If y' is not in T , compute $y'' = H(R(y'))$ and repeat the process.

Without hash chain (tutorial 1)

On input y , find w s.t. $H(w) = y$



On input y , find w s.t. $H(w) = y$



Analysis of time and space required.

Let's compare the space and time required compare to the simple method of using the full table.

Space: Reduction of space by a factor of 3. (why?)

Time per query: Number of hash operations increases by a factor of 3. Also require 2 reduce operations. (why?)

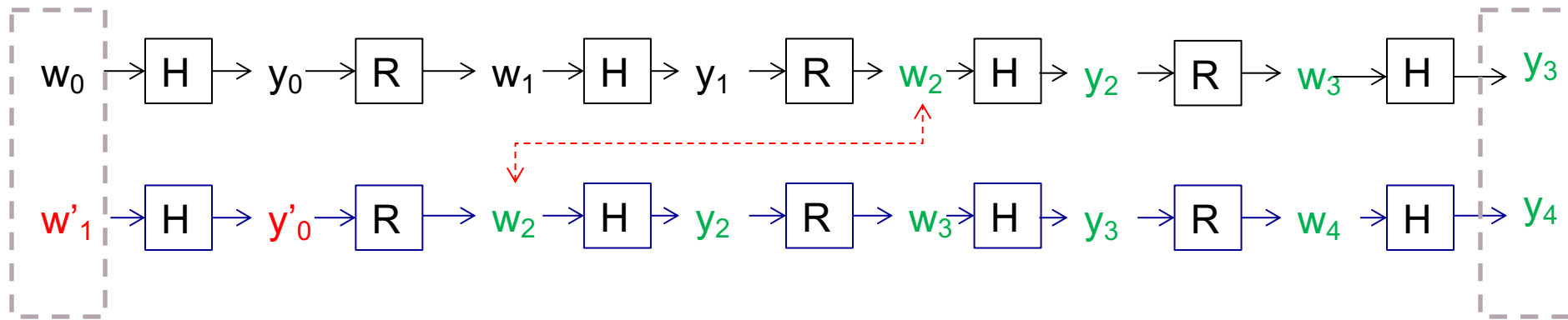
Efficiency: The chains contain repetitions. (why?)

In general, we can choose the length of the chain so that the reduction of space is a factor of k , with the penalty of increase the search time by a factor of k , where k is a parameter.

In our 50-bits example, if we choose $k=2^{15}$, and the number of entries in the table to 2^{35} , then the total number of entries covered in the “virtual table” is 2^{50} (as we shall see later, these 2^{50} entries are not unique). The query time increase to 2^{15} hash operations, which still can be computed in realtime, and the storage reduced to 2^{35} entries.

Efficiency Limitation: Duplicate in table due to Reduce function Collisions

- Collisions of the reduce function may happen frequently, i.e. two different digests being mapped to the same word.



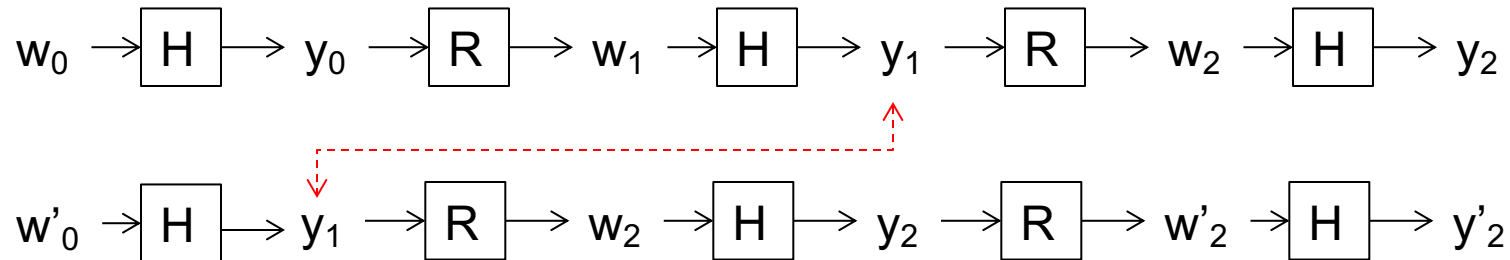
When given y'_0 , by following the chain w_0 will not lead to the correct w'_1 .

This causes two issues:

- Efficiency in storage: Part of the chain is duplicated. (w_2, w_3 appeared twice)
- Efficiency in search: Might need to transverse a few chain before hitting the correct one. (for the query y'_0 , the lookup process might need to transverse both w_0 and w'_1 chains).

Remark: Collisions due to Hash function is unlikely

- The following collision due to $H()$ is extremely unlikely (since we assume that $H()$ is “secure”) and thus can be omitted in our design consideration.



Improved variant: Rainbow Table

Rainbow table gives a simple but effective method to address the collision issue in time-memory tradeoff. (method is simple but analysis quite complex).

Rainbow table utilizes multiple “reduce” functions. Details not included in this module. To find out more more:

Reference:

http://en.wikipedia.org/wiki/Rainbow_table

<http://kestas.kuliukas.com/RainbowTables/>

The original research paper:

P. Oechslin. *Making a Faster Cryptanalytical Time-Memory Trade-Off*, CRYPTO 2003.

<http://lasec.epfl.ch/~oechslin/publications/crypto03.pdf>